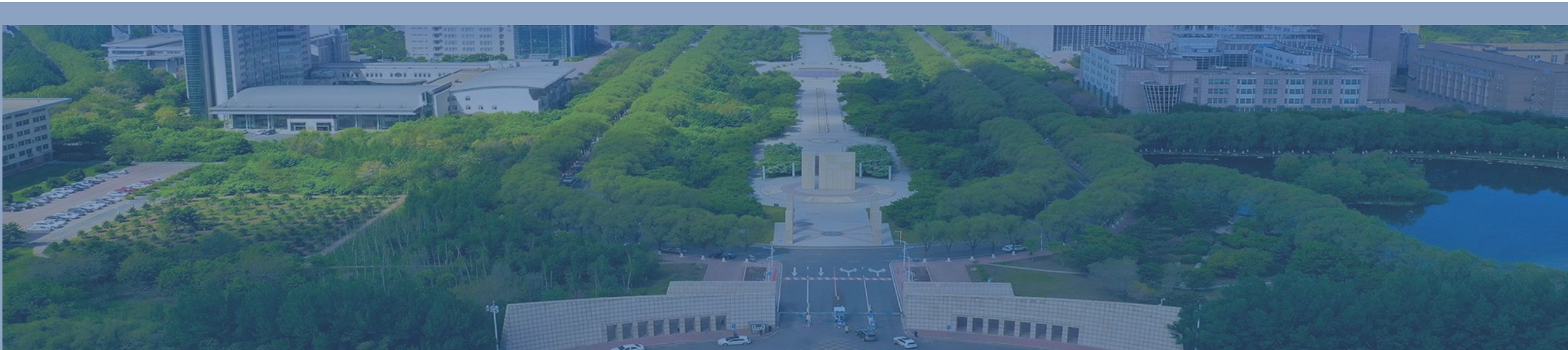




第六章 动态规划





目录

6.1 一般方法

6.2 多段图

6.3 货郎担问题

6.4 0/1背包问题

6.5 可靠性设计

6.6 最优二分检索树

6.7 矩阵链乘积问题

6.8 小结



6.1 一般方法

- 方法适用的问题特点
- 最优性原理
- 设计递推关系式
- 基于递推关系式实现程序
- 动态规划的设计思想
- 再谈动态规划的迭代实现
- 动态规划的特点



方法适用的问题特点

- 有一类多阶段决策问题
 - 它可以被分成若干个阶段，每一个阶段都需要作出决策， i 阶段的决策仅依赖于 i 阶段当前的状态
 - 当每个阶段的决策都确定后，得到一个决策序列，进而得到问题的一个解。
- 动态规划方法适用于解决**求最优解的多阶段决策**问题
 - 寻找最优解和最优决策序列

问题描述中一定存在目标函数

也称为多阶段决策优化问题，
或组合优化问题



动态规划的设计思想概要

20世纪50年代，贝尔曼等人根据多阶段决策问题的特性，提出了解决这类问题的“最优性原理”，从而创建了一种新的算法设计方法——**动态规划**。动态规划自问世以来，在经济管理、生产调度、工程技术和最优控制等领域得到了广泛应用。

- 动态规划方法求解多阶段决策问题的**步骤**：
 - 1) 证明多阶段决策(优化)问题满足最优性原理
 - 2) 设计递推关系式
 - 3) 迭代求解递推关系式



最优性原理(Principle of Optimality)

过程的最优决策序列具有如下性质： 无论过程的初始状态和初始决策是什么，其余的决策都必须相对于初始决策所产生的状态构成一个最优决策序列。

$$S_0 \xrightarrow{X_1} S_1 \xrightarrow{X_2} S_2 \xrightarrow{X_3} \dots \xrightarrow{X_n} S_n$$

原问题包含子问题，原问题的最优决策序列依赖于子问题的最优解。

- 问题满足最优性原理，则称其具有**最优子结构性质**。
- 最优性原理证明思想
 - ① 确定最优决策序列的形式
 - ② 确定初始状态和初始决策
 - ③ 确定初始决策所产生的状态
 - ④ 证明其余决策相对于③是最优决策序列



以0/1背包问题为例

- 问题描述：对于 n 个物品，容量为 M 的背包，要求物品或者整件装入背包中，或者根本不装入，即 x_i 限定只能取0或1值。

$$\begin{aligned} & \text{极大化} \quad \sum_{1 \leq i \leq n} p_i x_i \\ & \text{约束条件} \quad \sum_{1 \leq i \leq n} w_i x_i \leq M \\ & x_i = 0 \text{ 或 } 1, \quad 1 \leq i \leq n \end{aligned}$$

英文字母 L

- 用 $\text{KNAP}(l, j, X)$ 来表示这个问题，函数值表示最大效益和。对于 n 个物品，容量为 M 的背包，就可表示为 $\text{KNAP}(1, n, M)$

问题最优解

问题边界

约束条件



0/1背包问题的最优性原理证明

- 设 $x_1, x_2, \dots, x_n$ 是最优序列。
- 若 $x_1=0$ ，则 $x_2, x_3, \dots, x_n$ 必须相对于 $\text{KNAP}(2, n, M)$ 问题构成一个最优序列。如若不然，则 $x_1, x_2, \dots, x_n$ 就不是 $\text{KNAP}(1, n, M)$ 的最优序列。
- 若 $x_1=1$ ，则 $x_2, x_3, \dots, x_n$ 必须是 $\text{KNAP}(2, n, M-w_1)$ 的最优序列。如若不然，则必有另一个0/1序列 $z_2, z_3, \dots, z_n$ 使得 $\sum w_i z_i \leq M-w_1 (2 \leq i \leq n)$ 且 $\sum p_i z_i > \sum p_i x_i (2 \leq i \leq n)$ 。因此，序列 $x_1, z_2, z_3, \dots, z_n$ 是一个对问题 $\text{KNAP}(1, n, M)$ 具有更大效益值的序列。这与假设相矛盾。
- 所以0/1背包问题的最优性原理成立。



设计递推关系式

- 向前处理法(forward approach)
 - 从最后阶段开始,逐步向前递推,即根据 $x_{i+1}, \dots, x_n$ 的最优决策序列来列出求取 x_i 决策值的关系式。
- 向后处理法(backward approach)
 - 从初始阶段开始,逐步向后递推,即根据 $x_1, \dots, x_{i-1}$ 的最优决策序列来列出求取 x_i 决策值的关系式。



0/1背包问题向前处理法的递推关系式

- 根据最优性原理：

$$\text{KNAP}(1, n, M) = \max\{\text{KNAP}(2, n, M), \text{KNAP}(2, n, M - w_1) + p_1\}$$

参数 n 不起作用，舍弃后简化函数

- 设 $g_j(X) = \text{KNAP}(j+1, n, X)$ ，则 $g_0(M) = \text{KNAP}(1, n, M)$

上式简化为： $g_0(M) = \max\{g_1(M), g_1(M - w_1) + p_1\}$

- 推知一般： $g_j(X) = \max\{g_{j+1}(X), g_{j+1}(X - w_{j+1}) + p_{j+1}\}$

$$g_n(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$



0/1背包问题向后处理法的递推关系式

- 设 $f_i(X) = \text{KNAP}(1, i, X)$, 则 $f_n(M) = \text{KNAP}(1, n, M)$
- 那么 $f_n(M)$ 的递归关系式可表示为: $f_n(M) = \max \{ f_{n-1}(M), f_{n-1}(M - w_n) + p_n \}$
- 对于任意的 $f_i(X)$, $i > 0$ 有以下公式:

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X - w_i) + p_i \}$$

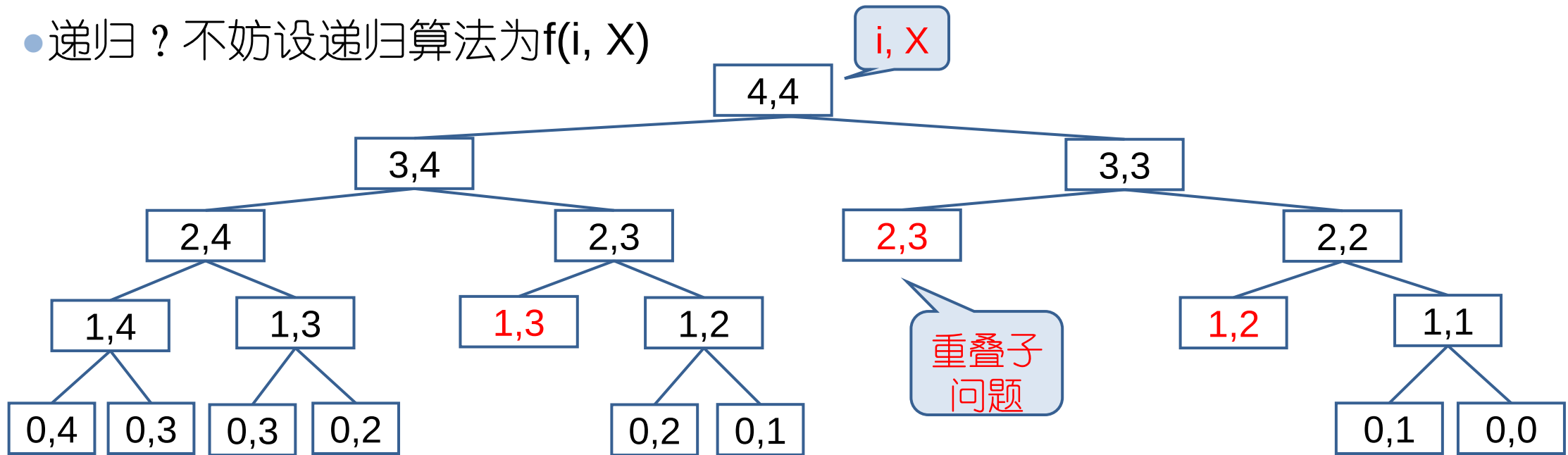
$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

思考：如何设计算法？

递推关系式（自顶向下递归）在设计程序时，采用自底向上迭代求解。

递推关系式求解

- 向后处理递推关系式: $f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$
- 例: $n=4, (p_1, p_2, p_3, p_4)=(1, 2, 5, 6), (w_1, w_2, w_3, w_4)=(1, 1, 1, 1), M=4$
 - 递归? 不妨设递归算法为 $f(i, X)$



- **重叠子问题性质**: 求解一个问题时, 同一个子问题被多次重用。
- 为提高效率, 自底向上迭代求解, 并保存子问题的求解结果。重叠子问题越多, 效率提升越明显。

动态规划的设计思想

- 动态规划方法求解问题时：
 - 证明多阶段决策(优化)问题满足最优性原理
 - 设计递推关系式
 - 设计函数：函数值代表问题最优解；参数集代表问题边界和约束条件
 - 根据最优性原理确定原问题和子问题的关系
 - 自底向上方式设计程序
 - 从最小的子问题开始迭代计算，直到达到原始问题规模
 - 计算过程中保存子问题的最优解和最优决策

刻画问题规模

备忘录

标记函数

针对一些问题，动态规划法比蛮力法(穷举)在时间复杂度上得到了实质的改进

再谈动态规划的迭代实现

- 自底向上计算子问题：
 - 每个子问题只计算一次。即计算出所有规模为1的子问题，再计算所有规模为2的子问题，……，直到规模n为止。
- 定义“**备忘录**” (和标记函数)记录子问题的最优解和最优决策：
 - 即定义一个存储空间，通常是表格形式，保存每个子问题的最优解(和最优决策)，记录最优解的表格称为“备忘录”。
 - 重用子问题时，直接查询备忘录。



小结——动态规划的特点

- 特点
 - 最优子结构性质(满足最优性原理)
 - 重叠子问题性质
- 优点
 - 设计思想优于蛮力法，实现方法优于递归法
- 缺点
 - 如果子问题重叠程度低，动态规划的时间复杂度改进不明显
 - 空间需求大



6.2 多段图问题

- 问题描述
- 最优性原理证明
- 多段图向前处理递推关系式
- 向前处理的计算过程
- 向前处理算法及执行过程
- 多段图向后处理递推关系式
- 向后处理的计算过程
- 多段图的应用

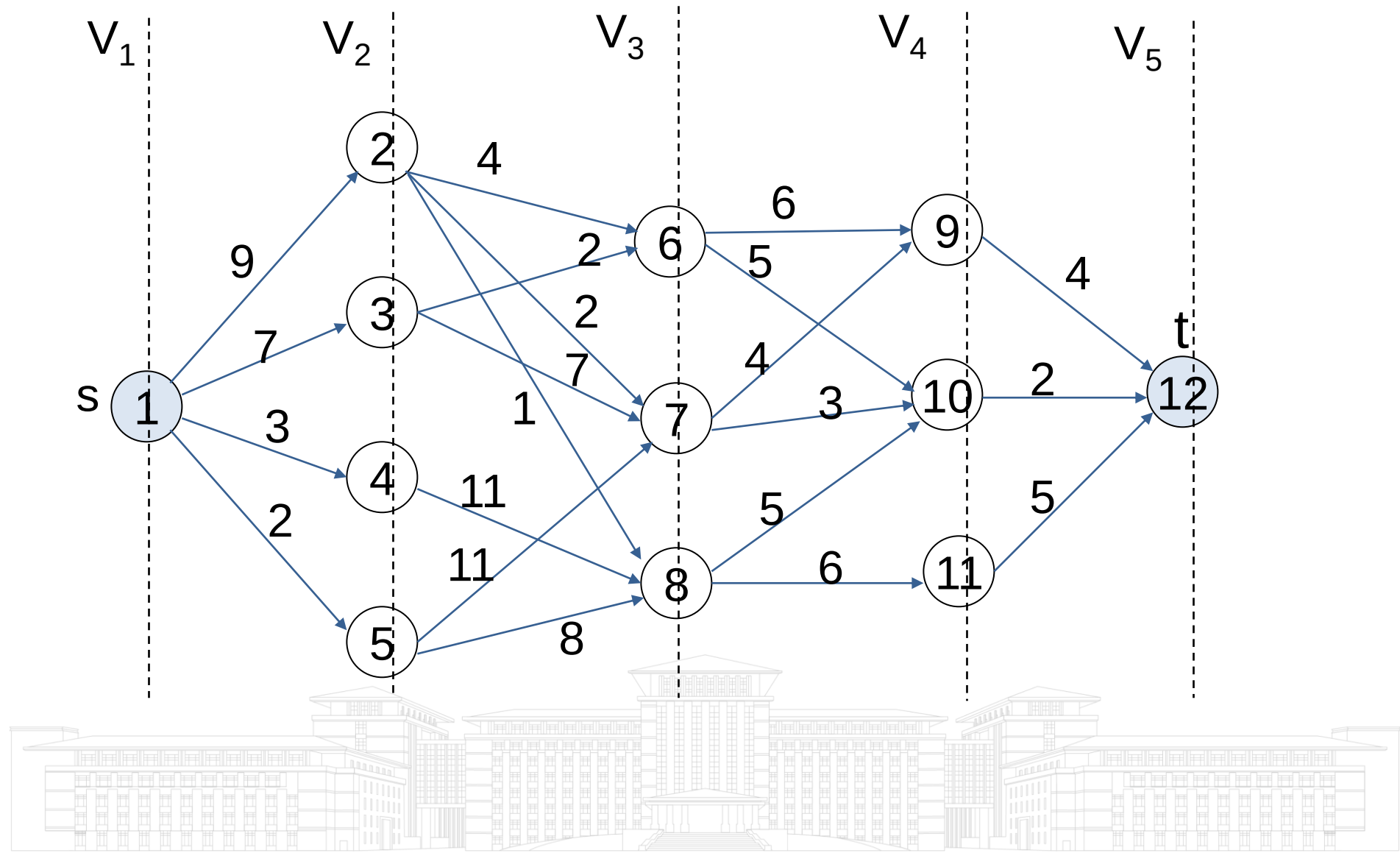


问题描述

- 多段图 $G=(V,E)$ 是一个有向图。它具有如下特性：
 - 图中的结点被划分成 $k \geq 2$ 个不相交的集合 V_i , $1 \leq i \leq k$, 其中 V_1 和 V_k 分别只有一个结点 s (源点) 和 t (汇点)
 - 图中所有的边 $\langle u, v \rangle$ 均具有如下性质: 若 $u \in V_i$, 则 $v \in V_{i+1}$, $1 \leq i < k$, 且每条边 $\langle u, v \rangle$ 均附有成本 $c(u, v)$
- 从 s 到 t 的一条路径成本是这条路径上边的成本和。
- 多段图问题是求由 s 到 t 的**最小成本**路径。



多段图示例



多段图问题的最优性原理证明

- 假设 $s, v_2, v_3, \dots, v_{k-1}, t$ 是一条由 s 到 t 的最短路径。
- 再假设从源点 s 开始，已作出了到结点 v_2 的决策， v_2 就是初始决策所产生的状态。
- 如果把 v_2 看成是原问题的一个子问题的初始状态，解决这个子问题就是找出一条由 v_2 到 t 的最短路径。
- 这条最短路径显然是 $v_2, v_3, \dots, v_{k-1}, t$ 。如果不然，设 $v_2, q_3, \dots, q_{k-1}, t$ 由 v_2 到 t 的更短路径，则 $s, v_2, q_3, \dots, q_{k-1}, t$ 肯定比 $s, v_2, v_3, \dots, v_{k-1}, t$ 路径短，这与假设矛盾。
- 因此最优性原理成立。



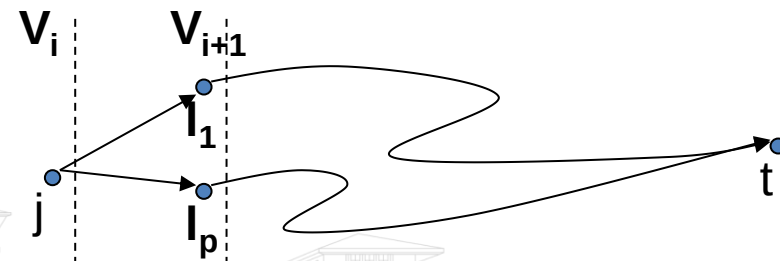
多段图向前处理递推关系式

- 设 $P(i, j)$ 是一条从 V_i 中的结点 j 到汇点 t 的最小成本路径, $COST(i, j)$ 表示这条路径的成本. 根据向前处理方法, 对于 j 的每条出边, 依次计算后取小:

$$COST(i, j) = \min\{c(j, l) + COST(i+1, l)\}$$

其中: j 的出边 $\langle j, l \rangle \in E, l \in V_{i+1}$, $c(j, l)$ 为该边的成本。

- 对于 k 段图, 当 $i=k-1$ 时(初值)
 - 如果 $\langle j, t \rangle \in E$, $COST(k-1, j) = c(j, t)$
 - 否则, $COST(k-1, j) = \infty$



最优解

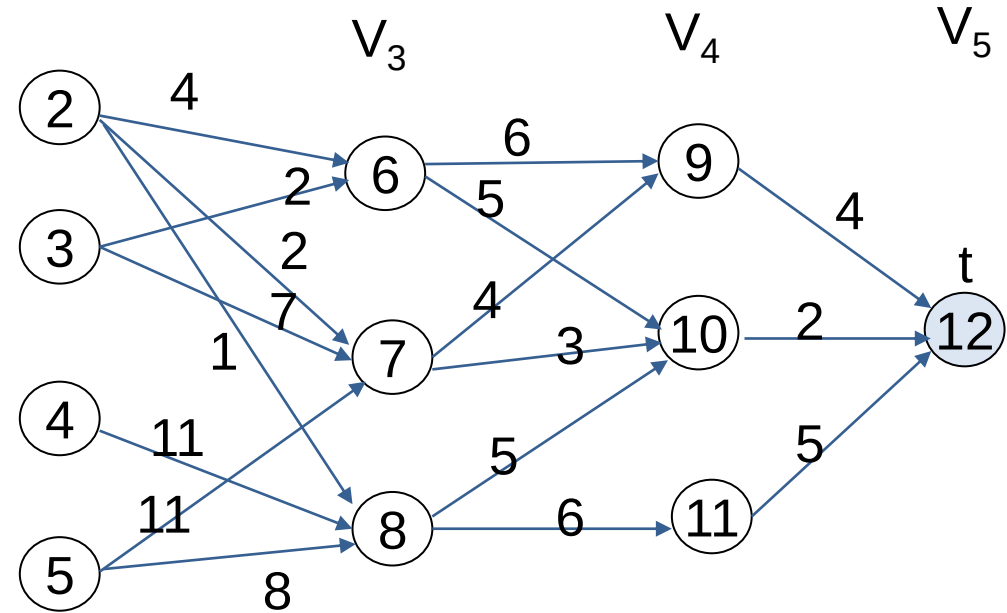
$$\text{COST}(i, j) = \min\{c(j, l) + \text{COST}(i+1, l)\}$$

从最小的子问题
开始迭代计算

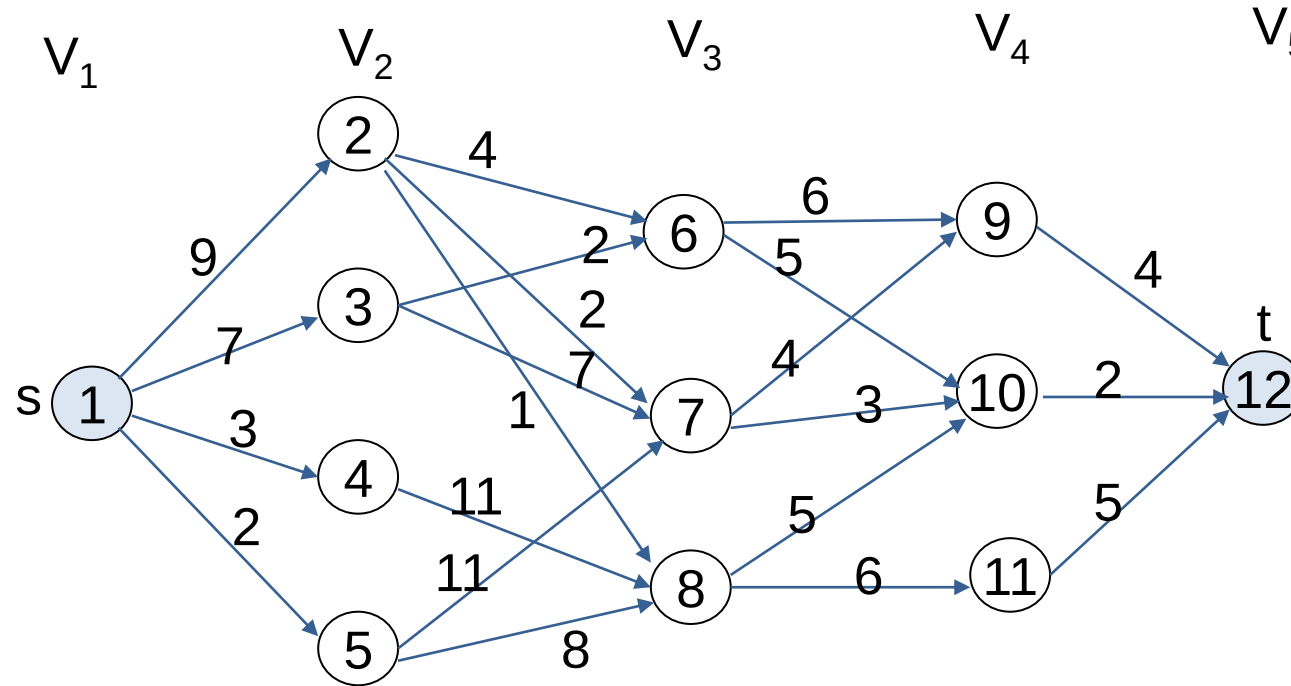
$$\begin{aligned} V_4 \quad & \text{COST}(4, 9) = c(9, 12) = 4 \\ & \text{COST}(4, 10) = c(10, 12) = 2 \\ & \text{COST}(4, 11) = c(11, 12) = 5 \end{aligned}$$

$$\begin{aligned} V_3 \quad & \text{COST}(3, 6) = \min\{c(6, 9) + \text{COST}(4, 9), c(6, 10) + \text{COST}(4, 10)\} = \min\{6 + 4, 5 + 2\} = 7 \\ & \text{COST}(3, 7) = \min\{c(7, 9) + \text{COST}(4, 9), c(7, 10) + \text{COST}(4, 10)\} = \min\{4 + 4, 3 + 2\} = 5 \\ & \text{COST}(3, 8) = \min\{c(8, 10) + \text{COST}(4, 10), c(8, 11) + \text{COST}(4, 11)\} = \min\{5 + 2, 6 + 5\} = 7 \end{aligned}$$

$$\begin{aligned} V_2 \quad & \text{COST}(2, 2) \\ & = \min\{c(2, 6) + \text{COST}(3, 6), c(2, 7) + \text{COST}(3, 7), c(2, 8) + \text{COST}(3, 8)\} = \min\{4 + 7, 2 + 5, 1 + 7\} \\ & = 7 \end{aligned}$$



V_2 $\text{COST}(2,2) = 7$
 $\text{COST}(2,3) = 9$
 $\text{COST}(2,4) = 18$
 $\text{COST}(2,5) = 15$



V_1 $\text{COST}(1,1) = \min\{c(1,2)+\text{COST}(2,2), c(1,3)+\text{COST}(2,3),$
 $c(1,4)+\text{COST}(2,4), c(1,5)+\text{COST}(2,5)\}$
 $= \min\{9+7, 7+9, 3+18, 2+15\} = 16$

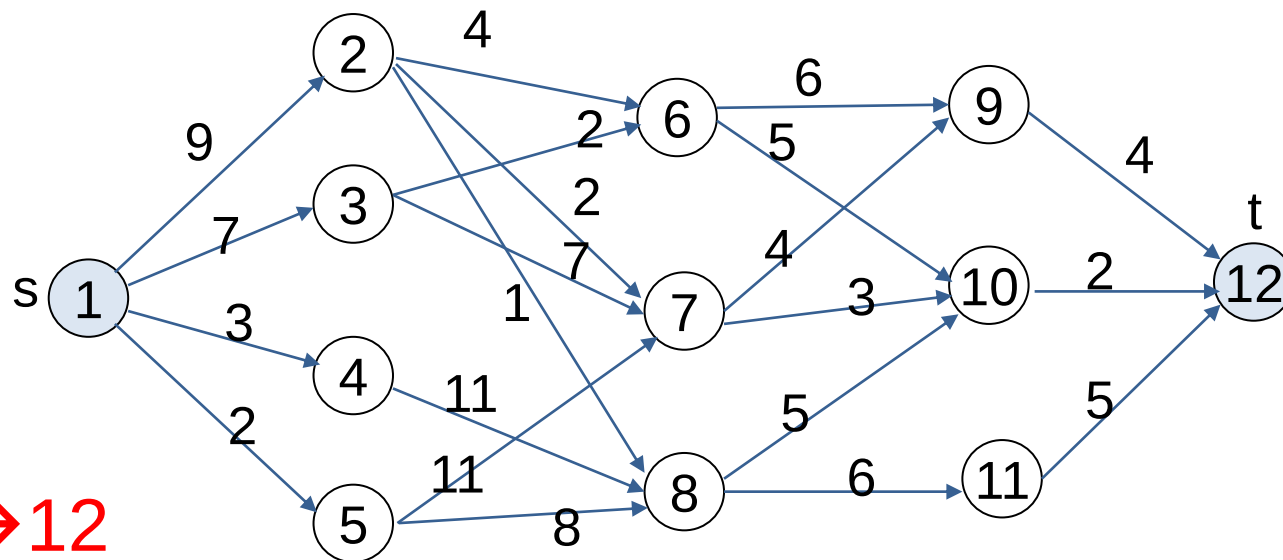
思考：为什么优于蛮力法(穷举)？



最优决策序列

- 在计算 $COST(i, j)$ 时, 记下每个状态(结点 j)所做出的决策 l , 令 $D(i, j) = l$, 由此求出最小成本的路径

V_1	V_2	V_3
$D(1, 1) = \underline{2}$	$D(2, 2) = \underline{7}$	$D(3, 6) = 10$
	$D(2, 3) = 6$	$D(3, 7) = \underline{10}$
	$D(2, 4) = 8$	$D(3, 8) = 10$
	$D(2, 5) = 8$	



最小成本的路径为: **1 → 2 → 7 → 10 → 12**

思考: 是否可以减低参数规模?

对结点进行编号, 从 V_1 开始 s 编为1号, 然后 V_2 中的结点依次编为2,3,4,5号, 按此规则编下去, 可以将 $COST$, D 中表示段数的第一个下标 i 省略掉。

算法6.1 多段图的向前处理算法

procedure FGRAPH(E,k,n,P) P(1:k)保存最小成本路径上节点，即最优决策序列

real COST(n), int D(n-1), P(k), r, j, k, n

COST(n) $\leftarrow$ 0

for j $\leftarrow$ n-1 to 1 by -1 do

寻找结点r, 满足 $\langle j, r \rangle \in E$ 且使 $c(j,r) + \text{COST}(r)$ 值最小

COST(j) $\leftarrow$ $c(j,r) + \text{COST}(r)$;

D(j) $\leftarrow$ r;

repeat

P(1) $\leftarrow$ 1; P(k) $\leftarrow$ n

for j $\leftarrow$ 2 to k-1 do P(j) $\leftarrow$ D(P(j-1)) repeat

end FGRAPH

- 时间复杂度: $\Theta(n+e)$
 - n是图G中节点个数。
 - e是图G中边的个数。
- 空间复杂度: $\Theta(n+e)$
 - 邻接表的存储空间
 - 除E外, 还需要COST、D、P

多段图可用邻接表存储

FGRAPH算法执行过程

COST(1)=16
D(1)=2

COST(5)=15
COST(4)=18
COST(3)=9
COST(2)=7
D(5)=8
D(4)=8
D(3)=6
D(2)=7

COST(8)=7
COST(7)=5
COST(6)=7
D(8)=10
D(7)=10
D(6)=10

COST(11)=5
COST(10)=2
COST(9)=4
D(11)=12
D(10)=12
D(9)=12

	1	2	3	4	5	6	7	8	9	10	11	12
COST	16	7	9	18	15	7	5	7	5	2	4	0
	1	2	3	4	5	6	7	8	9	10	11	
D	2	7	6	8	8	10	10	10	12	12	12	
	1	2	3	4	5							
P	1	2	7	10	12							

保存各阶段子问题的最优决策和最优解：

COST: 备忘录，存储解；

D: 标记函数，存储决策

多段图向后处理递推关系式

- 设 $BP(i, j)$ 是一条从源点 s 到 V_i 中的结点 j 的最小成本路径, $BCOST(i, j)$ 表示这条路径的成本, 根据向后处理方法:

$$BCOST(i, j) = \min\{BCOST(i-1, l) + c(l, j)\}$$

其中: j 的入边 $\langle l, j \rangle \in E, l \in V_{i-1}, c(l, j)$ 为该边的成本。

- 对于 k 段图, 当 $i=2$ 时(初值)
 - 如果 $\langle s, j \rangle \in E, COST(2, j) = c(s, j)$
 - 否则, $COST(2, j) = \infty$



最优解

$$\text{BCOST}(i, j) = \min\{\text{BCOST}(i-1, l) + c(l, j)\}$$

简化为:

$$\text{BCOST}(j) = \min\{\text{BCOST}(l) + c(l, j)\}$$

$$\text{BCOST}(2)=9$$

$$\text{BCOST}(3)=7$$

$$\text{BCOST}(4)=3$$

$$\text{BCOST}(5)=2$$

$$D(2)=1$$

$$D(3)=\underline{1}$$

$$D(4)=1$$

$$D(5)=1$$

$$\text{BCOST}(6)=9$$

$$\text{BCOST}(7)=11$$

$$\text{BCOST}(8)=10$$

$$D(6)=\underline{3}$$

$$D(7)=2$$

$$D(8)=2$$

$$\text{BCOST}(9)=15$$

$$\text{BCOST}(10)=14$$

$$\text{BCOST}(11)=16$$

$$D(9)=6$$

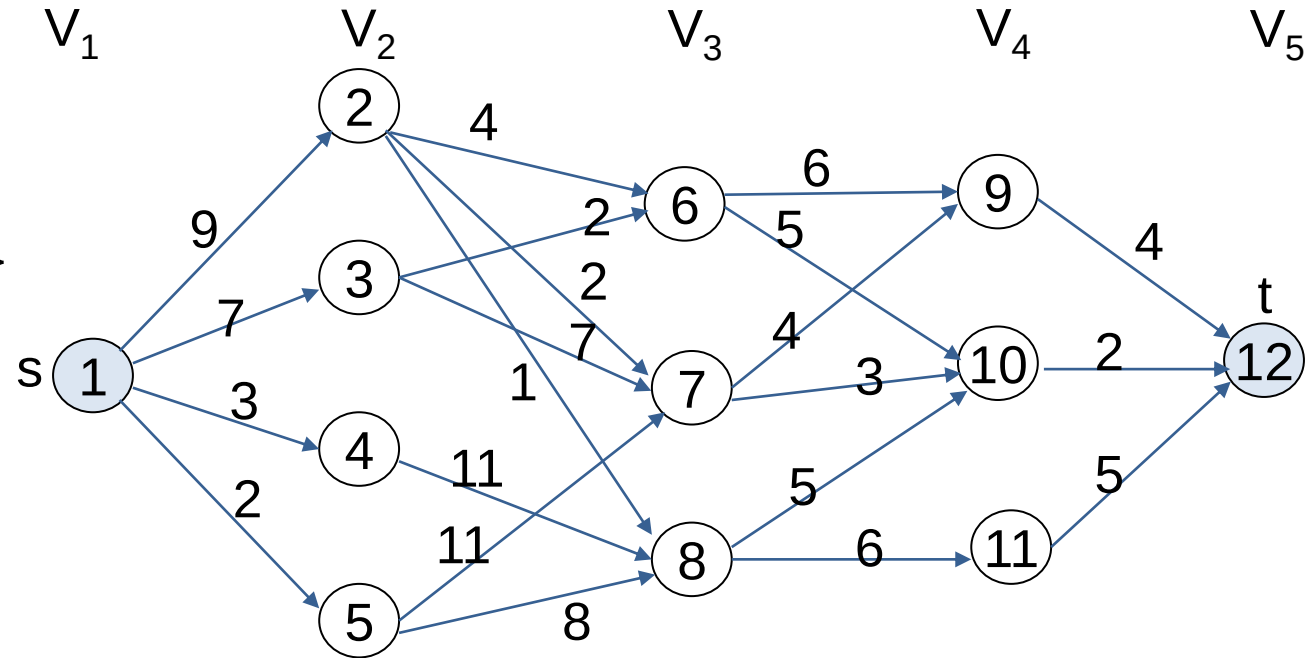
$$D(10)=\underline{6}$$

$$D(11)=8$$

$$\text{BCOST}(12)=16$$

$$D(12)=\underline{10}$$

最小成本的路径为: $1 \rightarrow 3 \rightarrow 6 \rightarrow 10 \rightarrow 12$



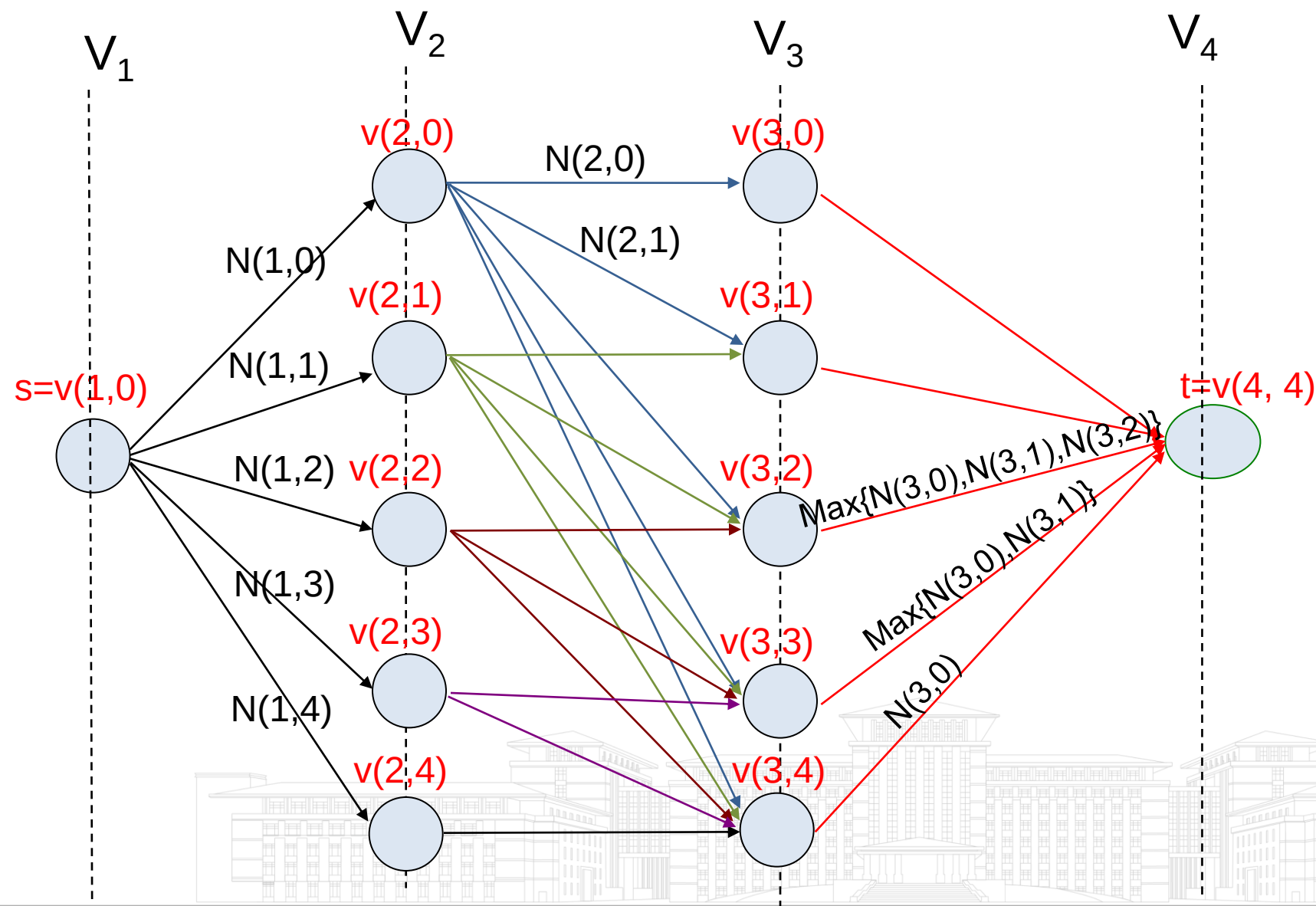
思考: 如何设计多段图向后处理算法? 向后处理算法中如何存储多段图?

多段图的应用——资源分配问题

- 多段图问题，模型简单、用处很大，很多实际问题可以用多段图来描述。
- 资源分配问题：给 r 个项目分配 n 个资源的问题。
 - 已知： $N(i, j)$ 表示给项目 i 分配 j 个资源所获得的利润 ($1 \leq i \leq r, 0 \leq j \leq n$)
 - 求：使总利润最大的分配方案
- 用多段图来表示该问题 ($r+1$ 段图)
 - 点 $V(i, j)$ 表示已给前 $i-1$ 个项目分配 j 个资源所对应的状态。
 - 边 $\langle V(i, j), V(i+1, l) \rangle$ 表示给项目 i 分配 $l-j$ 个资源所获得的利润 $N(i, l-j)$ 。当 $i=r$ 时，边 $\langle V(r, j), V(r+1, n) \rangle$ 的利润为 $\max\{N(r, p) | 0 \leq p \leq n-j\}$ 。
 - 求从 $V(1, 0)$ 到 $V(r+1, n)$ 的最长路径。



例：给3个项目最多分配4个资源的资源分配问题



- 边 $N(i,j)$ 是给项目 i 分配 j 个资源所获得的利润;
- $V(i,j)$ 表示在阶段 i 已分配 j 个资源后系统状态;
- s 到 t 的最长路径, 为最大利润分配方案。

6.3 货郎担问题

- 问题描述
- 问题分析
- 递推关系式
- 计算过程
- 时间复杂度分析

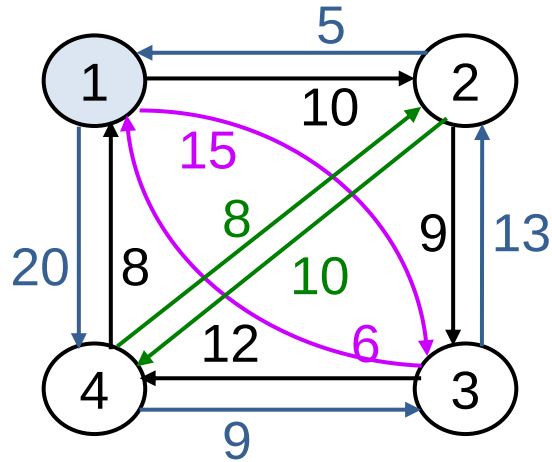


问题描述

- 货郎担问题：某售货员要到若干个村庄售货，各村庄之间的路程是已知的，为了提高效率，售货员决定从所在商店出发，到每个村庄售货一次，最后返回商店，问他应选择一条什么路线才能使所走的总路程最短。
- 类似问题
 - 邮车收集邮件的邮路问题
 - 机械手臂在装配线上的移动问题
 - 产品的生产安排问题



问题分析

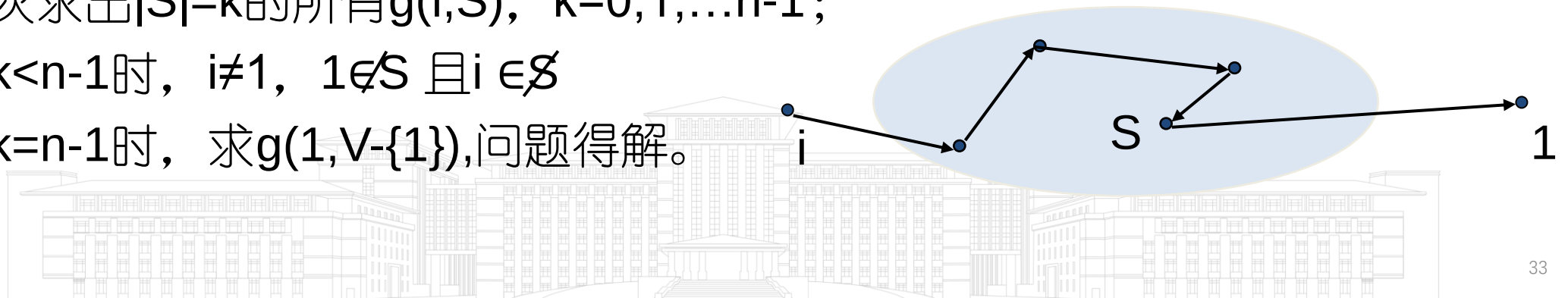


设 $G(V, E)$ 是一个具有边成本 c_{ij} 的有向图, G 的一条周游路线是包含 V 中每个结点的一个有向环, 周游路线的成本是此路线上所有边的成本之和。

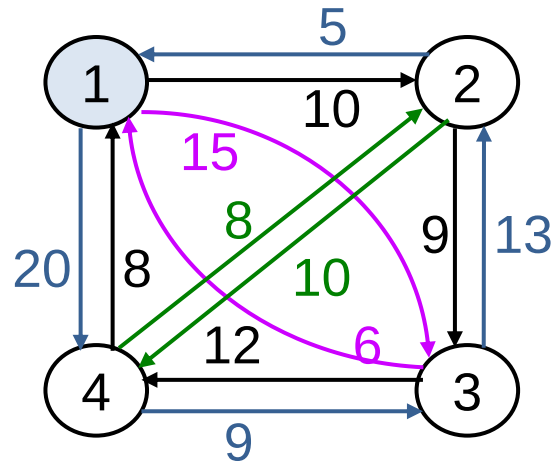
- 假设周游路线是开始于结点1并终止于结点1的一条简单路径。
 - 每一条周游路线都是由一条边 $\langle 1, k \rangle$ 和一条由结点 k 到结点1的简单路径所组成的, 其中 $k \in V - \{1\}$ 。
 - 这条由结点 k 到结点1的路径通过 $V - \{1, k\}$ 的每个结点各一次。
- 如果这条周游路线是最优的, 则这条由 k 到1的路径必定是通过 $V - \{1, k\}$ 中所有结点的由 k 到1的最短路径, 因此最优性原理成立

递推关系式

- 设 $g(i, S)$ 是由结点 i 开始，通过 S 中的所有结点，在结点 1 终止的一条最短路径长度。
 - $g(1, V-\{1\})$ 是一条最优的周游路线长度
 - 于是可以得到： $g(1, V-\{1\}) = \min_{2 \leq k \leq n} \{c_{1k} + g(k, V-\{1, k\})\}$
- 递推公式 $g(i, S) = \min_{j \in S} \{c_{ij} + g(j, S-\{j\})\}$
 初始值 $g(i, \emptyset) = c_{i1}, 1 < i \leq n$
- 求解过程
 - 依次求出 $|S|=k$ 的所有 $g(i, S)$, $k=0, 1, \dots, n-1$;
 - 当 $k < n-1$ 时, $i \neq 1, 1 \notin S$ 且 $i \in S$
 - 当 $k = n-1$ 时, 求 $g(1, V-\{1\})$, 问题得解。



计算过程



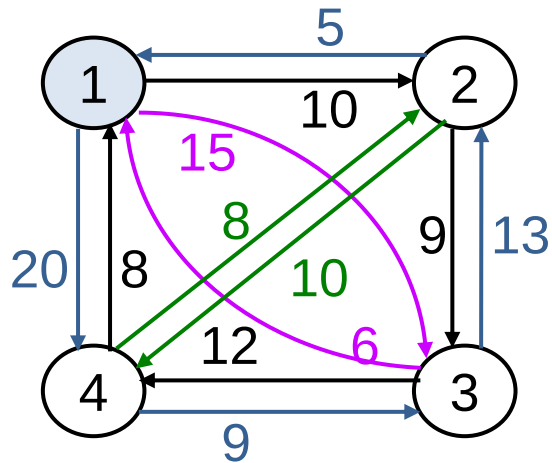
	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

$|S|=0$

- $g(2, \emptyset) = c_{21} = 5$
- $g(3, \emptyset) = c_{31} = 6$
- $g(4, \emptyset) = c_{41} = 8$

$|S|=1$

- $g(2, \{3\}) = c_{23} + g(3, \emptyset) = 9 + 6 = 15$
- $g(2, \{4\}) = c_{24} + g(4, \emptyset) = 10 + 8 = 18$
- $g(3, \{2\}) = c_{32} + g(2, \emptyset) = 13 + 5 = 18$
- $g(3, \{4\}) = c_{34} + g(4, \emptyset) = 12 + 8 = 20$
- $g(4, \{2\}) = c_{42} + g(2, \emptyset) = 8 + 5 = 13$
- $g(4, \{3\}) = c_{43} + g(3, \emptyset) = 9 + 6 = 15$



	1	2	3	4
1	0	10	15	20
2	5	0	9	10
3	6	13	0	12
4	8	8	9	0

$$|S|=2 \quad \bullet \quad g(2, \{3, 4\}) = \min\{c_{23} + g(3, \{4\}), c_{24} + g(4, \{3\})\} \\ = \min\{9 + 20, \mathbf{10 + 15}\} = \mathbf{25}$$

$$\bullet \quad g(3, \{2, 4\}) = \min\{c_{32} + g(2, \{4\}), c_{34} + g(4, \{2\})\} \\ = \min\{13 + 18, \mathbf{12 + 13}\} = \mathbf{25}$$

$$\bullet \quad g(4, \{2, 3\}) = \min\{c_{42} + g(2, \{3\}), c_{43} + g(3, \{2\})\} \\ = \min\{\mathbf{8 + 15}, 9 + 18\} = \mathbf{23}$$

$$|S|=3 \quad \bullet \quad g(1, \{2, 3, 4\}) = \min\{c_{12} + g(2, \{3, 4\}), c_{13} + g(3, \{2, 4\}), \\ c_{14} + g(4, \{2, 3\})\} \\ = \min\{\mathbf{10 + 25}, 15 + 25, 20 + 23\} = \mathbf{35}$$

可得这条最优周游路线是: $1 \rightarrow 2 \rightarrow 4 \rightarrow 3 \rightarrow 1$

思考1: 从2出发的最优周游路线是哪一条?

思考2: 和贪心法相比为什么能够保证获得最优解?

时间复杂度分析

- $|S|=k$, $k=0,1,\dots,n-2$ 时, 对于 k 的每一种取值, $g(i,S)$ 中的结点 i 都有 $n-1$ 种选择 (即 $i=2,3,\dots,n$), 对应每个 i 的 S 的个数为 C_{n-2}^k , S 是 $V-\{1,i\}$ 的子集。

k 的每一种取值对应的函数求解次数为: $(n-1)C_{n-2}^k$

k 的所有取值对应的函数求解次数为: $\sum_{k=0}^{n-2} (n-1)C_{n-2}^k = (n-1)2^{n-2}$

又知求函数 $g(i,S)$ 的值要进行 $|S|-1$ 比较: $\sum_{k=0}^{n-2} (n-1)(k-1)C_{n-2}^k = \Theta(n^2 2^n)$

优于枚举法的结果 $O(n!)$,
但随着 n 的增大, 该方法亦不可取。



6.4 0/1背包问题

- 问题描述和递推关系式
- 迭代算法——算法6.2
- 序偶法——图解、序偶法思路与步骤、复杂度分析、算法改进



问题描述和递推关系式

- 0/1背包问题要求物品或者整件装入背包中, 或者根本不装入(即不能装入物品的一部分), 所以 x_i 限定只能取0或1值。

$$\begin{array}{ll} \text{极大化} & \sum_{1 \leq i \leq n} p_i x_i \\ \text{约束条件} & \sum_{1 \leq i \leq n} w_i x_i \leq M \end{array} \quad \begin{array}{l} x_i = 0 \text{ 或 } 1, \quad p_i > 0 \\ w_i > 0, \quad 1 \leq i \leq n \end{array}$$

- 向后处理的递推关系式

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

思考：0/m背包问题是指 x_i 取值范围是自然数，如何形式化描述问题和设计递推关系式？

递推关系式计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5), M=6$
- 备忘录 $F(i, X)$ 记录子问题的解

$F(i, X)$	$X=0$	1	2	3	4	5	$6=M$
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2							
3							

初值

$$f_1(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty + 1\} = 0 & 0 \leq X < 2 \\ \max\{0, 0 + 1\} = 1 & 2 \leq X \leq 6 \end{cases}$$



递推关系式计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5), M=6$
- 备忘录 $F(i, X)$

$F(i, X)$	$X=0$	1	2	3	4	5	$6=M$
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	2	2	3	3
3							

$$f_2(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ \max\{1, 0+2\}=2 & 3 \leq X < 5 \\ \max\{1, 1+2\}=3 & 5 \leq X \leq 6 \end{cases}$$



递推关系式计算过程

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- 实例： $n=3, (w_1, w_2, w_3)=(2, 3, 4), (p_1, p_2, p_3)=(1, 2, 5), M=6$
- 备忘录 $F(i, X)$

$F(i, X)$	$X=0$	1	2	3	4	5	$6=M$
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	2	2	3	3
3	0	0	1	2	5	5	6

$$f_3(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ 2 & 3 \leq X < 4 \\ 5 & 4 \leq X < 6 \\ 6 & 6 \leq X < 7 \end{cases}$$

最优解

备忘录的大小与 n 和 M 有关，为什么 X 在 $[0, M]$ 范围内取值？

回溯确定X

- 实例： $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$, $(p_1, p_2, p_3)=(1, 2, 5)$, $M=6$
- x_3 : $M=X=6$, $F(3,6) \neq F(2,6)$, 则 $x_3=1$, $X-w_3=6-4=2$
- x_2 : $F(2,2)=F(1,2)$, 则 $x_2=0$
- x_1 : $F(1,2) \neq F(0,2)$, 则 $x_1=1$

$F(i, X)$	$X=0$	1	2	3	4	5	$6=M$
0	0	0	0	0	0	0	0
1	0	0	1	1	1	1	1
2	0	0	1	2	2	3	3
3	0	0	1	2	5	5	6



算法6.2 0/1背包算法描述

```
procedure KNAPSACK(p,w,n,M)
int p(n), w(n), F(0:n,0:M), x(1:n), i, j, n //F是备忘录
for j ← 0 to M do F(0,j) ← 0 repeat
for i ← 1 to n do
    for j ← 0 to M do
        F(i,j) ← F(i-1,j)
        if w(i) ≤ j and F(i,j) < F(i-1,j-w(i))+p(i) then
            F(i,j) ← F(i-1,j-w(i))+p(i)
        endif
    repeat
repeat //最优解在F(n,M)中
    回溯确定x的值
end FGRAPH
```

- 时间复杂度: $\Theta(nM)$
 - n是物品个数
 - M是背包容量
- 空间复杂度: $\Theta(nM)$

伪多项式时间

思考: 可以认为0/1背包问题是多项式时间的算法么?

```

procedure KNAPSACK1(p,w,n,M)
int p(n), w(n), F(0:M), x(1:n), i, j, n
for j ← 0 to M do F(j) ← 0 repeat
for i ← 1 to n do
    for j ← M to w(i) do //倒序更新F
        if F(j) < F(j-w(i))+p(i) then
            F(j) ← F(j-w(i))+p(i)
        endif
    repeat
repeat //最优解在F(M)中
回溯确定x的值
end FGRAPH
    
```

- 为了降低空间复杂度，可否将备忘录 $F(0:n,0:M)$ 去掉子问题边界 i ，改为一维 **$F(0:M)$** ，空间复杂度从 $\Theta(nM)$ 降低到 $\Theta(M)$ ？此时，时间复杂度没有变化： **$\Theta(nM)$**
- 若这样做，可以得到最优解，但不能通过回溯得到最优决策序列。

• 实例： $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$, $(p_1, p_2, p_3)=(1, 2, 5)$, $M=6$

F(j)	0	1	2	3	4	5	6
初始	0	0	0	0	0	0	0
<u>i</u> =1时	0	0	1	1	1	1	1
<u>i</u> =2时	0	0	1	2	2	3	3
<u>i</u> =3时	0	0	1	2	5	5	6

i=n时，F(M)是问题最优解

递推关系式图解

$$f_i(X) = \max \{ f_{i-1}(X), f_{i-1}(X-w_i)+p_i \}$$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

- $f_{i-1}(X-w_i)+p_i$ 的图像可由 $f_{i-1}(X)$ 在x轴上右移 w_i 个单位，然后上移 p_i 个单位得到。
- $f_i(X)$ 的图像由 $f_{i-1}(X)$ 和 $f_{i-1}(X-w_i)+p_i$ 的函数曲线按X相同时取最大值的规则归并而成。



实例: $n=3$, $(w_1, w_2, w_3)=(2, 3, 4)$, $(p_1, p_2, p_3)=(1, 2, 5)$, $M=6$

$$f_0(X) = \begin{cases} -\infty & X < 0 \\ 0 & X \geq 0 \end{cases}$$

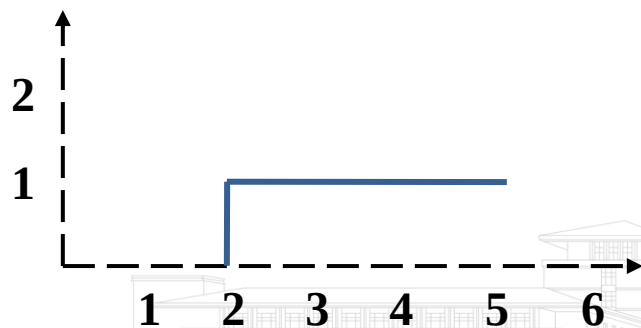
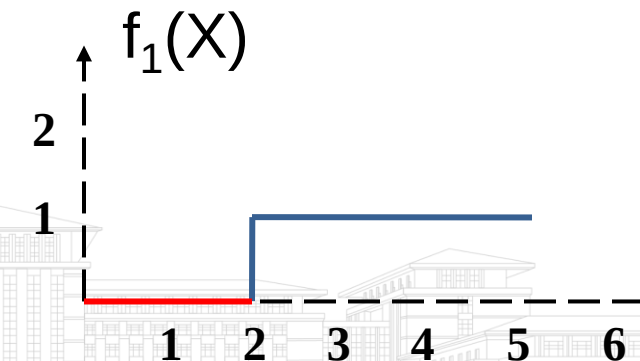
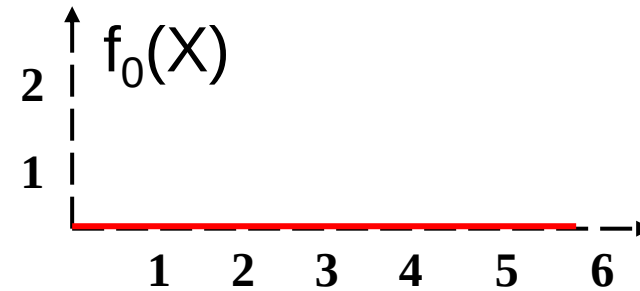
$$f_1(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty + 1\} = 0 & 0 \leq X < 2 \\ \max\{0, 0 + 1\} = 1 & X \geq 2 \end{cases}$$

$$f_{i-1}(X-w_i)+p_i$$

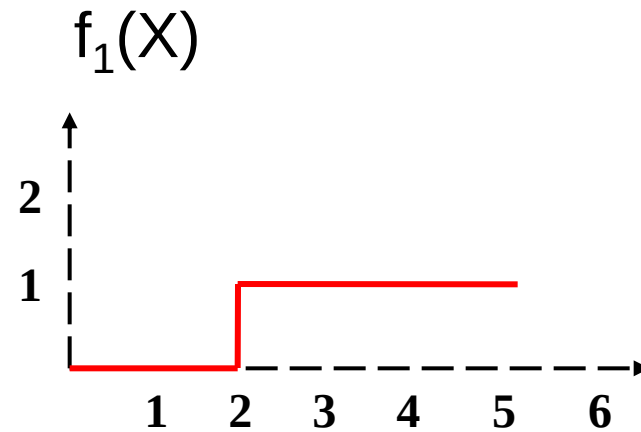
$$f_i(X)$$

$i=0$: 函数不存在

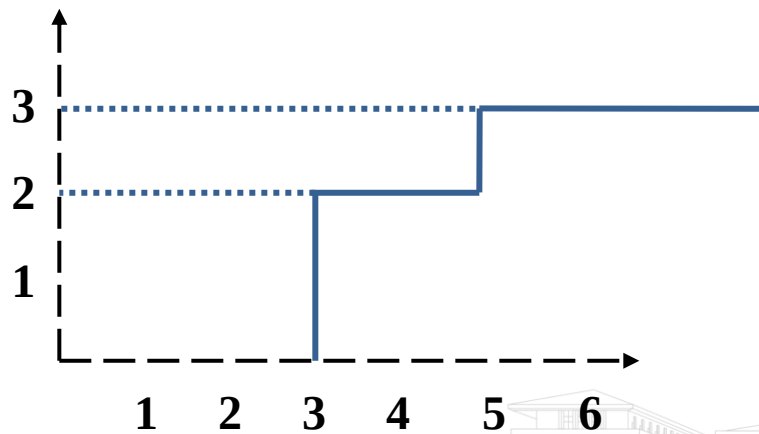
$$i=1: f_0(X-2)+1$$



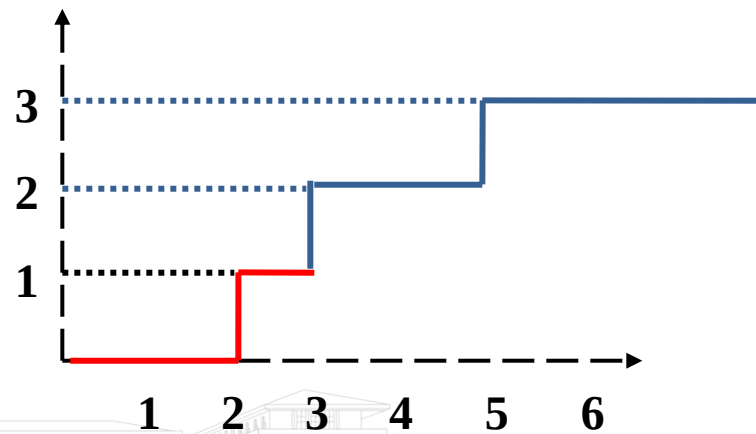
$$f_2(X) = \begin{cases} -\infty & X < 0 \\ 0 & 0 \leq X < 2 \\ 1 & 2 \leq X < 3 \\ \max\{1, 0+2\}=2 & 3 \leq X < 5 \\ \max\{1, 1+2\}=3 & 5 \leq X \leq 6 \end{cases}$$



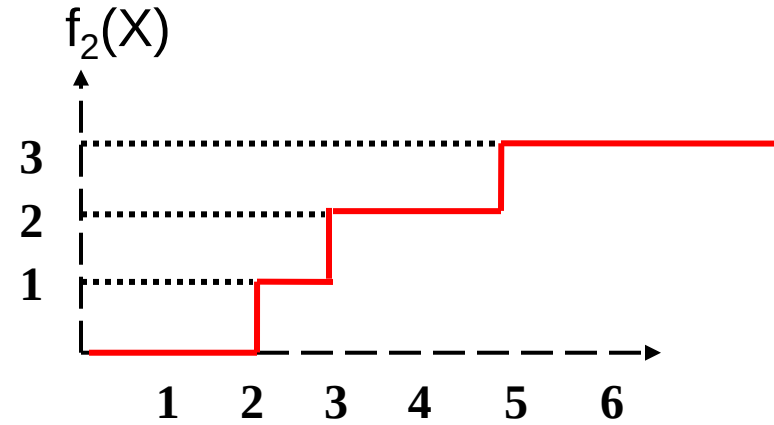
$i=2: f_1(X-3)+2$



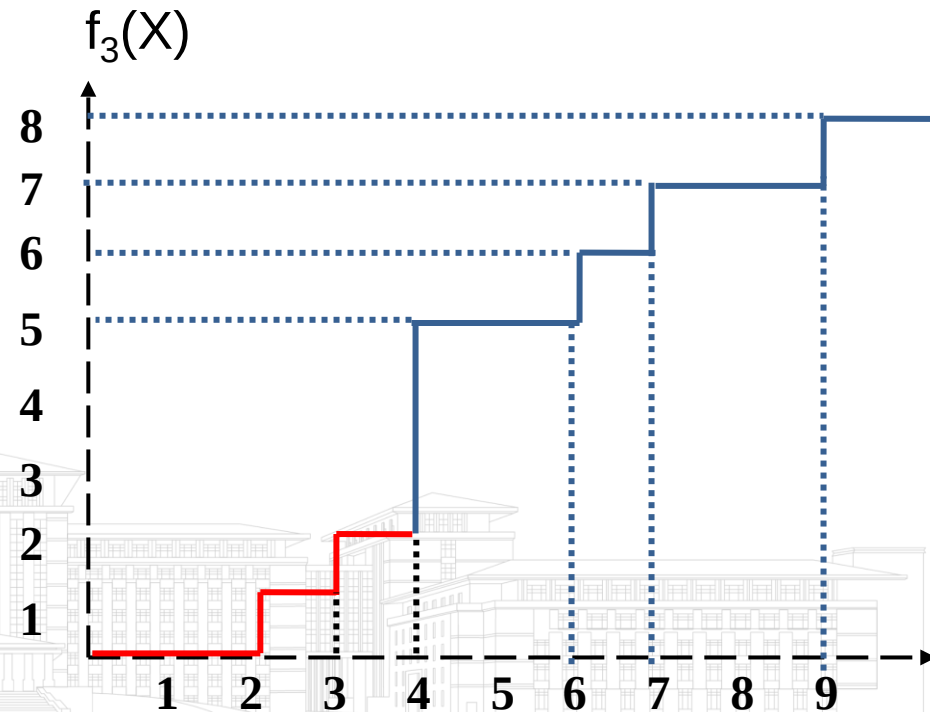
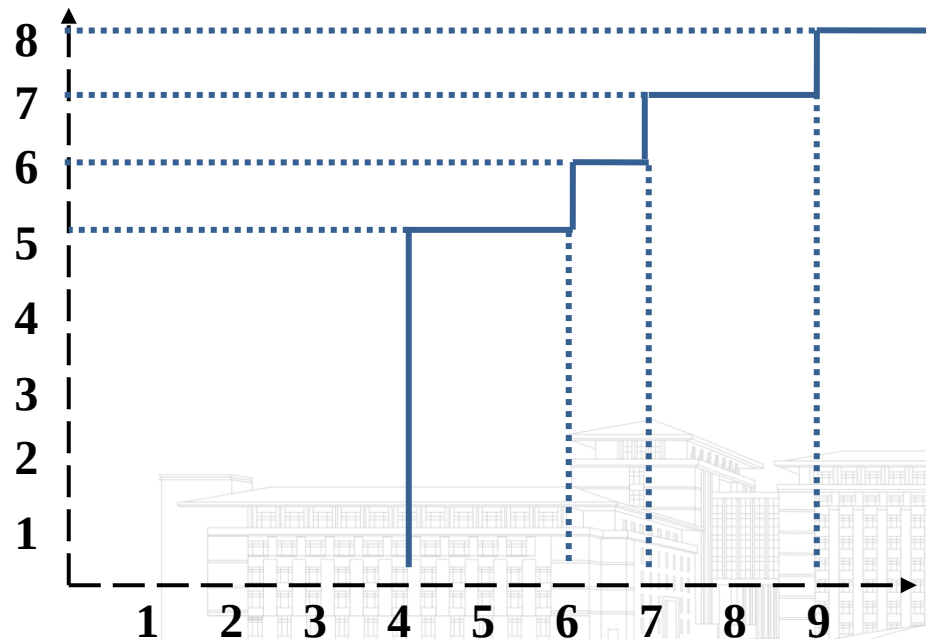
$f_2(X)$



$$f_3(X) = \begin{cases} -\infty & X < 0 \\ \max\{0, -\infty + 5\} = 0 & 0 \leq X < 2 \\ \max\{1, -\infty + 5\} = 1 & 2 \leq X < 3 \\ \max\{2, -\infty + 5\} = 2 & 3 \leq X < 4 \\ \max\{2, 0 + 5\} = 5 & 4 \leq X < 5 \\ \max\{3, 0 + 5\} = 5 & 5 \leq X < 6 \\ \max\{3, 1 + 5\} = 6 & 6 \leq X < 7 \\ \max\{3, 2 + 5\} = 7 & 7 \leq X < 9 \\ \max\{3, 3 + 5\} = 8 & X \geq 9 \end{cases}$$

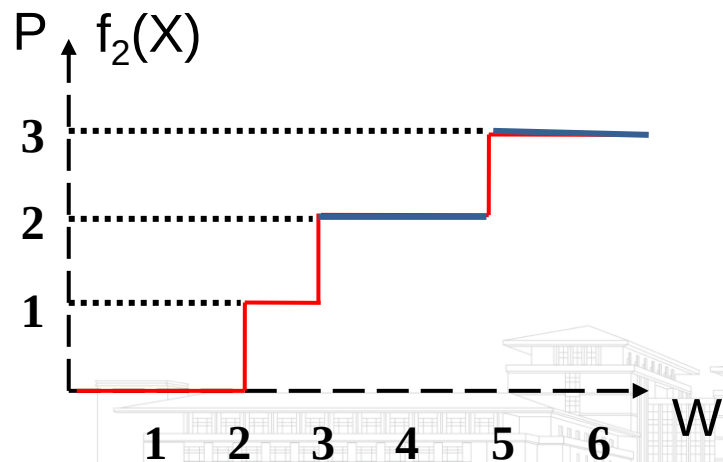


$$i=3: f_2(X-4)+5$$



函数曲线分析

- 每一个 f_i 完全可由一些序偶 (P_j, W_j) 组成的集合说明, 其中 W_j 是使 f_i 在其处产生一次阶跃的 X 值, $P_j=f_i(W_j)$ 。
- 第一对序偶 $(P_0, W_0)=(0,0)$; 如果有 r 次阶跃, 就还有 r 对序偶 (P_j, W_j) , $1 \leq j \leq r$; 在 r 对序偶中, 假定 $W_j < W_{j+1}$, 则有 $P_j < P_{j+1}$ 。
- 在 $0 \leq j < r$ 情况下, 对所有满足 $W_j \leq X < W_{j+1}$ 的 X , 有 $f_i(X)=f_i(W_j)$; 对所有满足 $X \geq W_r$ 的 X , 有 $f_i(X)=f_i(W_r)$ 。



3次阶跃, f_2 由序偶 $(0,0), (1,2), (2,3), (3,5)$ 组成的集合所刻画

$3 \leq X < 5$, $f_i(X)=f_i(3)=2$

$X \geq 5$, 有 $f_i(X)=f_i(5)=3$

序偶集合定义

- S^{i-1} : f_{i-1} 的所有序偶的集合
- S_1^i : $f_{i-1}(X-w_i)+p_i$ 的所有序偶的集合, 将 (p_i, w_i) 加到 S^{i-1} 的每一对序偶上就得到 S_1^i
- S^i : S^{i-1} 和 S_1^i 基于支配规则实现归并

支配规则: 已知 $(P_j, W_j) \in S^{i-1}$, $(P_k, W_k) \in S_1^i$, 并且在 $W_j \geq W_k$ 的同时有 $P_j \leq P_k$, 那么序偶 (P_j, W_j) 被放弃, 称为 (P_k, W_k) 支配 (P_j, W_j) 。



序偶法设计思想

- $S^0 = \{(0,0)\}$
- $i \geq 1$ 时, 将 (p_i, w_i) 加到 S^{i-1} 的每一对序偶上就得到 S_1^i ,
即 $S_1^i = \{(p, w) | (p - p_i, w - w_i) \in S^{i-1}\}$
- 在支配规则下将 S^{i-1} 和 S_1^i 归并成 S^i 。
- 在生成 S^i 时, 将 $w > M$ 的那些序偶 (p, w) 去掉, 因为它们不能产生满足约束条件的可行解。

S^i 中的序偶表示对于不同的约束 $X (X \leq M)$, 前 i 个物品可以取得的最大效益值。

S^n 中最末序偶的 P 值, 是问题的最优解。

序偶法实例

实例: $n=3$, $(p_1, p_2, p_3)=(1, 2, 5)$, $(w_1, w_2, w_3)=(2, 3, 4)$, $M=6$

$$S^0=\{(0,0)\} \xrightarrow{(p_1, w_1)=(1,2)} S^1_1=\{(1,2)\}$$

$$S^1=\{(0,0), (1,2)\} \xrightarrow{(p_2, w_2)=(2,3)} S^2_1=\{(2,3), (3,5)\}$$

$$S^2=\{(0,0), (1,2), (2,3), (3,5)\} \xrightarrow{(p_3, w_3)=(5,4)}$$

$$S^3_1=\{(5,4), (6,6), (7,7), (8,9)\}$$

$w > M$, 舍弃掉

根据支配规则, $(5,4)$
支配掉 $(3,5)$

$$S^3=\{(0,0), (1,2), (2,3), (5,4), (6,6)\}$$



确定决策序列

- 对于 S^n 的最末序偶 (p_l, w_l) , 若 $(p_l, w_l) \in S^{n-1}$, 则 $x_n=0$; 若 $(p_l, w_l) \notin S^{n-1}$, 且 $(p_l - p_n, w_l - w_n) \in S^{n-1}$, 则 $x_n=1$
- 再判断留在 S^{n-1} 的序偶 (p_l, w_l) 或 $(p_l - p_n, w_l - w_n)$ 是否属于 S^{n-2} 以确定 x_{n-1} 的取值
- ...

$$S^0 = \{(0,0)\}$$

$$S^1 = \{(0,0), (1,2)\}$$

$$x_1=1$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\}$$

$$x_2=0$$

$$S^3 = \{(0,0), (1,2), (2,3), (5,4), (6,6)\}$$

$$x_3=1$$

最优解 $f_n(M)$ 的最优决策序列是 $(x_1, x_2, x_3) = (1, 0, 1)$

实现序偶法的数据结构

- $p(1:n)$: 物品的效益值。
- $w(1:n)$: 物品的重量。
- $P(1:m)$ 和 $W(1:m)$: 存放序偶集合 S , 集合元素对应序偶 (p_i, w_i) ; 且集合 $S^0, S^1, \dots, S^{n-1}$ 互相邻接的存放。
- $F(0:n)$: 指针数组, 元素 $F(i)$ 指示 S^i 中的第一个元素所在的位置。

$$S^0 = \{(0,0)\}$$

$$S^1 = \{(0,0), (1,2)\}$$

$$S^2 = \{(0,0), (1,2), (2,3), (3,5)\}$$

	1	2	3	4	5	6	7	8
P	0	0	1	0	1	2	3	
W	0	0	2	0	2	3	5	

$F(0)$ $F(1)$ $F(2)$ $F(3)$

序偶法的空间复杂度分析

- S^i 的序偶数量用 $|S^i|$ 表示，每个 S^i 由 S^{i-1} 和 S^i_1 归并而成，故 $|S^i| \leq 2|S^{i-1}|$
- 最坏情况下，归并过程中没有序偶被清除，即

$$|S^0|=1, \quad |S^1|=2, \quad |S^2|=2+2=2^2, \quad \dots, \quad |S^{n-1}|=2^{n-1}$$

$$\sum_{0 \leq i \leq n-1} |S^i| = \sum_{0 \leq i \leq n-1} 2^i = 2^n - 1$$

- 即空间复杂度是 $O(2^n)$



序偶法的时间复杂度分析

- 由 S^{i-1} 生成 S^i ，需要的时间是 $\Theta(|S^{i-1}|)$ ；计算所有的 $S^0, S^1, \dots, S^{n-1}$ ，需要的总时间为 $\Theta(\sum |S^{i-1}|)$ ；因为 $|S^i| \leq 2^i$ ，故总时间为 $O(2^n)$
- 若 P, W 均为整数，因没有重复序偶，于是有 $|S^i| \leq 1 + \sum_{1 \leq j \leq i} P_j$ ， $|S^i| \leq 1 + \min\{\sum_{1 \leq j \leq i} W_j, M\}$
- 故当 P 和 W 为整数时，时间复杂度为

$$O(\min\{2^n, n \sum_{1 \leq j \leq n} P_j, nM\})$$

M 一般远小于 2^n ，再加上支配规则，能解决 n 较大时的问題。



序偶法的改进或优化

- 思路：除了支配规则及约束条件，试图进一步删除不可能达到最优解的序偶。
- 试探法：设 L 为最优解的估计值，即 $f_n(M) \geq L$ ；定义 $PLEFT(i) = \sum_{i < j \leq n} p_j$ ，如果 $(p, w) \in S^i$ ，且 $p + PLEFT(i) < L$ ，那么将 (p, w) 从 S^i 中清除。

实例： $n=6$ ， $M=165$

$$\begin{aligned}
 & (p_1, p_2, p_3, p_4, p_5, p_6) \\
 &= (w_1, w_2, w_3, w_4, w_5, w_6) \\
 &= (100, 50, 20, 10, 7, 3),
 \end{aligned}$$

- $S^0 = \{0\}$, $S^1_1 = \{100\}$, $S^1 = \{0, 100\}$, 因为 $0 + PLEFT(1) < L$;
- $S^2_1 = \{150\}$, $S^2 = \{100, 150\}$, 因为 $100 + PLEFT(2) < L$;
- $S^3_1 = \emptyset$, 不满足约束, $S^3 = \{150\}$,
- $S^4_1 = \{160\}$, $S^4 = \{150, 160\}$, 因为, $150 + PLEFT(4) < L$;
- $S^5_1 = \emptyset$, 不满足约束, $S^5 = \{160\}$
- $S^6_1 = \{163\}$, $S^6 = \{163\}$

假设贪心法求出估值 $L=163$ （也可其它方法估值）

	0	1	2	3	4	5	6
PLEFT(i)	190	90	40	20	10	3	0

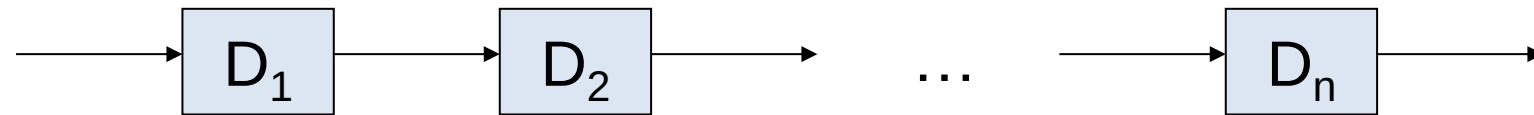
6.5 可靠性设计

- 问题描述
- 可靠性设计最优化问题
- 递推关系式
- 可靠性算法设计思想
- 问题实例计算过程



问题描述

- 假定设计一个系统，该系统由若干个以串联方式连接在一起的不同设备所组成。



- 设 r_i 是设备 D_i 正常运转的概率，即可靠性。

$\prod r_i$ 是整个系统的可靠性。

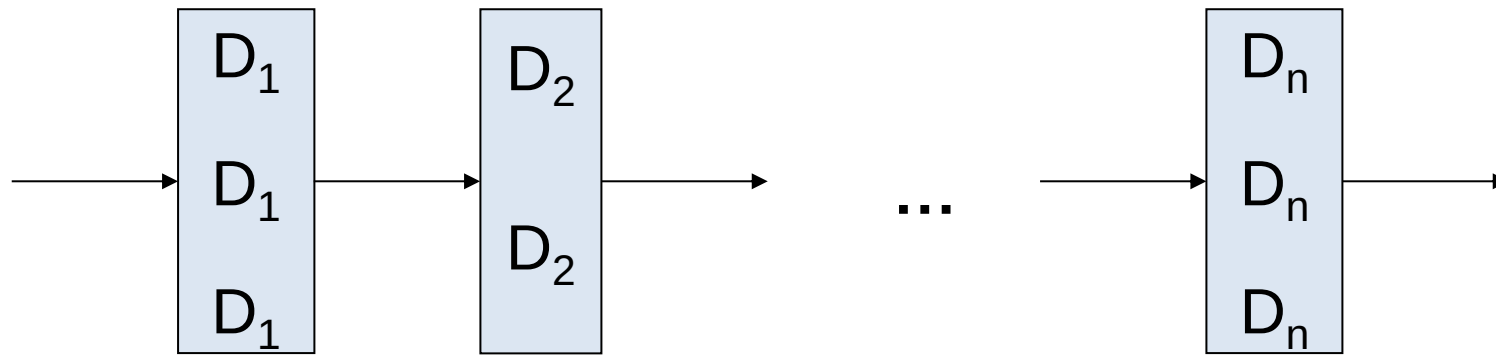
可靠性数据对比：

$$r_i=0.99, i=1,2,\dots,10 \quad \prod r_i = 0.99^{10} = 0.904$$

$$r_1=0.9, r_2=0.8, r_3=0.5, \quad \prod r_i = 0.36$$

改进方法

- 增加一些重复的设备。



- 假设设备 D_i 的台数为 m_i ，那么它们同时出现故障的概率为 $(1-r_i)^{m_i}$ ，则该级设备的可靠性变为 $1-(1-r_i)^{m_i}$ 。

例如： $r_i=0.99$ ， $m_i=2$ ，则可靠性： $1-(1-0.99)^2=0.9999$



可靠性设计最优化问题

- 问题描述：在最大成本 c 的约束下，如何使系统的可靠性达到最优的问题。
- 已知系统中每种设备至少有一台，设 c_i 是一台设备 i 的成本，则设备 i 允许配置的台数至多为：

$$u_i = 1 + \left\lfloor (c - \sum_{k=1}^n c_k) / c_i \right\rfloor$$

- 第 i 种设备的可靠性为函数： $\phi_i(m_i) = 1 - (1 - r_i)^{m_i}$ $1 \leq m_i \leq u_i$



递推关系式

- 用 $RELI(1,i,X)$ 表示在可容许成本 X 约束下，对第1种到第 i 种设备的可靠性设计问题。

$$\begin{array}{ll} \text{极大化} & \prod_{1 \leq j \leq i} \phi_j(m_j) \\ \text{约束条件} & \sum_{1 \leq j \leq i} c_j m_j \leq X \quad 1 \leq m_j \leq u_j \end{array}$$

- $f_n(c) = \max_{1 \leq m_n \leq u_n} \{ \phi_n(m_n) f_{n-1}(c - c_n m_n) \}$

- $f_i(x) = \max_{1 \leq m_i \leq u_i} \{ \phi_i(m_i) f_{i-1}(x - c_i m_i) \}$

- 使用类似 **0/1** 背包问题的序偶法来求解



可靠性设计问题算法思想

支配规则:对于 (f_1, x_1) 和 (f_2, x_2) , 当且仅当 $f_1 \geq f_2$ 而 $x_1 \leq x_2$ 时, (f_1, x_1) 支配 (f_2, x_2) 。
 (f_2, x_2) 从序偶集合中舍去。

- 令 $S^0 = \{(1, 0)\}$

- 基于 S^{i-1} 构造 S_j^i :

对于 $(f, x) \in S^{i-1}$, 有 $(\phi_i(j) \times f, x + c_i \times j) \in S_j^i$,

其中, $1 \leq j \leq u_i$, $x + c_i \times j + \sum_{i+1 \leq k \leq n} c_k \leq c$

- 基于支配规则归并: $S^i = \bigcup_{1 \leq j \leq u_i} S_j^i$

- 算法步骤:

先求每个 u_i , $i=1, 2, \dots, n$;

for $i \leftarrow 1$ to n

for $j \leftarrow 1$ to u_i

求 $\phi_i(j)$

求 S_j^i

repeat

基于支配规则归并 S^i

repeat



问题实例计算过程

- 设计一个由设备 D_1, D_2, D_3 组成的三级系统。每台设备的成本分别为30元，15元和20元，可靠性分别是0.9, 0.8, 0.5，计划建立该系统的投资不得超过105元。

$$c=105, c_1=30, c_2=15, c_3=20$$

$$r_1=0.9, r_2=0.8, r_3=0.5$$

- S^i : 表示 $m_1, \dots, m_i$ 的各种决策序列产生的所有不受支配的序偶 (f, x) 集合。
- S_j^i : 表示第 i 级设备选择 j 台时 $m_1, \dots, m_{i-1}, j$ 的各种决策序列产生的所有不受支配的序偶 (f, x) 集合。



- $S^0 = \{(1, 0)\}$
- $u_1 = \lfloor (105 - 15 - 20) / 30 \rfloor = 2, u_2 = 3, u_3 = 3$
- $u_1 = 2, c_1 = 30, r_1 = 0.9,$

$$\phi_1(1) = 0.9, \phi_1(2) = 1 - (1 - 0.9)^2 = 0.99$$

$$S^1_1 = \{(0.9, 30)\}, S^1_2 = \{(0.99, 60)\}$$

$$S^1 = \{(0.9, 30), (0.99, 60)\}$$

- $u_2 = 3, c_2 = 15, r_2 = 0.8,$

$$\phi_2(1) = 0.8, \phi_2(2) = 1 - (1 - 0.8)^2 = 0.96, \phi_2(3) = 1 - (1 - 0.8)^3 = 0.992$$

$$S^2_1 = \{(0.9 * 0.8, 30 + 15), (0.99 * 0.8, 60 + 15)\} = \{(0.72, 45), (0.792, 75)\}$$

$$S^2_2 = \{(0.9 * 0.96, 30 + 15 * 2), (0.99 * 0.96, 60 + 15 * 2)\} = \{(0.864, 60)\}$$

$$S^2_3 = \{(0.9 * 0.992, 30 + 15 * 3), (0.99 * 0.992, 60 + 15 * 3)\} = \{(0.8928, 75)\}$$

$$S^2 = \{(0.72, 45), (\text{red } 0.792, \text{red } 75), (0.864, 60), (0.8928, 75)\}$$

不满足约束条件

支配规则

- $S^2 = \{(0.72, 45), (0.864, 60), (0.8928, 75)\}$
- $u_3 = 3, c_3 = 20, r_3 = 0.5,$
 $\phi_3(1) = 0.5, \phi_3(2) = 1 - (1 - 0.5)^2 = 0.75, \phi_3(3) = 1 - (1 - 0.5)^3 = 0.875$
 $S^3_1 = \{(0.72 * 0.5, 45 + 20), (0.864 * 0.5, 60 + 20), (0.8928 * 0.5, 75 + 20)\}$
 $= \{(0.36, 65), (0.432, 80), (0.4464, 95)\}$
 $S^3_2 = \{(0.72 * 0.75, 45 + 20 * 2), (0.864 * 0.75, 60 + 20 * 2), (0.8928 * 0.75, 75 + 20 * 2)\}$
 $= \{(0.54, 85), (0.648, 100)\}$
 $S^3_3 = \{(0.72 * 0.875, 45 + 20 * 3)\} = \{(0.63, 105)\}$
 $S^3 = \{(0.36, 65), (0.432, 80), (0.4464, 95), (0.54, 85), (0.648, 100), (0.63, 105)\}$
- 由 $S^n, S^{n-1} \dots S^1$ 回溯求 $m_n, m_{n-1}, \dots, m_1$
- 判断由 S^n 中 f 最大的序偶来自哪个 S^n_j , 则 n 级设备上的设备数量为 j , 以此类推:
- $(0.648, 100) \in S^3_2$, 所以 $m_3 = 2$; $(0.864, 60) \in S^2_2$, 所以 $m_2 = 2$; $(0.9, 30) \in S^1_1$, $m_1 = 1$
 决策序列 $(m_1, m_2, m_3) = (1, 2, 2)$

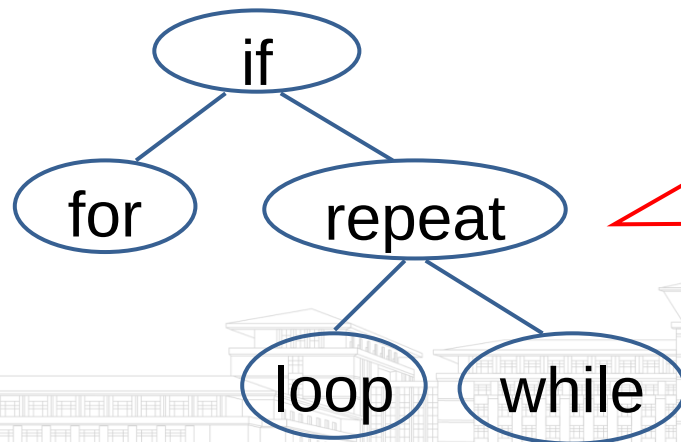
6.6 最优二分检索树

- 二分检索树
- 构造最优二分检索树问题描述
- 递推关系式分析
- 问题求解过程
- 时间复杂度分析
- D.E.Knuth的优化方法



二分检索树

- 二分检索树: T 是一个二叉树, 它或者为空, 或者其每个结点含有一个可比较大小的数据元素, 并且:
 - T 的左子树的所有元素比根结点 T 中的元素小
 - T 的右子树的所有元素比根结点 T 中的元素大
 - T 的左、右子树也是二分检索树



二分检索树中的所有结点的元素是互异的。

二分检索树检索算法

- 将X与根比较:
 - 若X比根中的元素小则检索左子树
 - 若X等于根中的元素则检索成功终止
 - 若X比根中的元素大则检索右子树

每个结点有3个信息段,
LCHILD, RCHILD, IDENT。
若X不在T中, 则 $i=0$; 否则,
将 i 置成使得 $IDENT(i)=X$ 。

```
procedure SEARCH(T, X, i)
  i ← T
  while i ≠ 0 do
    case
      :X < IDENT(i): i ← LCHILD(i)
      :X = IDENT(i): return
      :X > IDENT(i): i ← RCHILD(i)
    endcase
  repeat
end SEARCH
```



二分检索树的检索成本

- 设给定的标识符集是 $(a_1, a_2, \dots, a_n)$, $a_1 < a_2 < \dots < a_n$, $P(i)$ 是对 $X = a_i$ 成功检索的概率, $Q(i)$ 是 $a_i < X < a_{i+1}$ 不成功检索的概率(假定 $a_0 = -\infty$, $a_{n+1} = +\infty$),

$$\sum_{1 \leq i \leq n} P(i) + \sum_{0 \leq i \leq n} Q(i) = 1$$

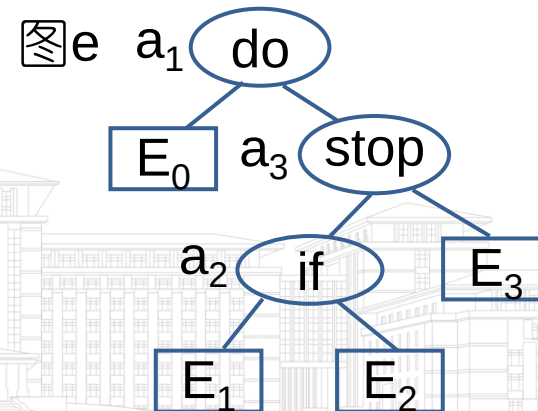
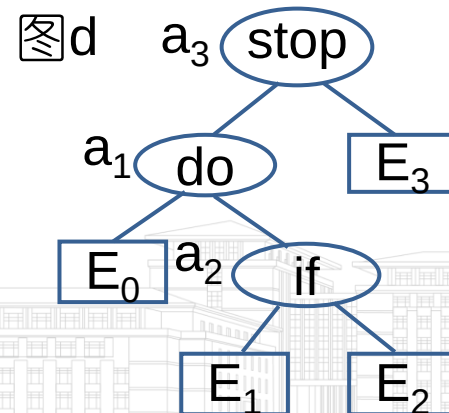
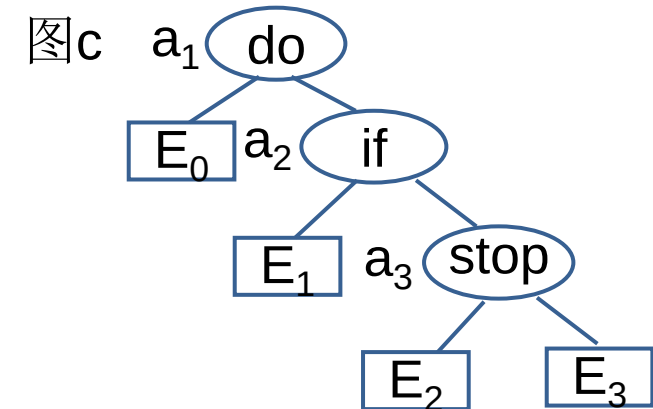
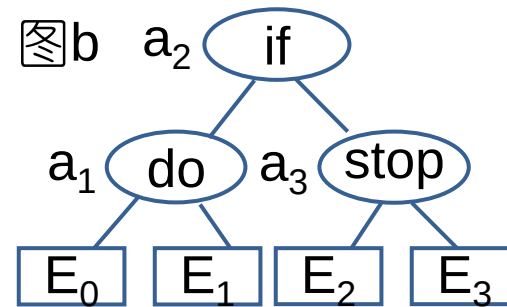
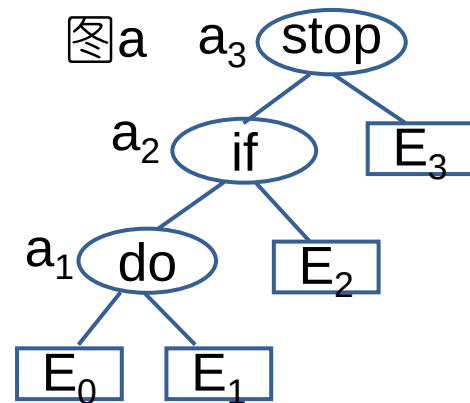
- 一次成功检索在一个内结点(l 级)处终止, 算法中的循环要执行 l 次, 所以内结点 a_i 的预期成本为 $P(i) * \text{level}(a_i)$
- 一次不成功检索在一个外结点处(l 级)终止, 算法中循环要执行 $l-1$ 次, 所以外结点 E_i 的预期成本为 $Q(i) * (\text{level}(E_i) - 1)$

二分检索树的预期检索成本为: $\sum_{1 \leq i \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$



检索成本与哪些因素有关？

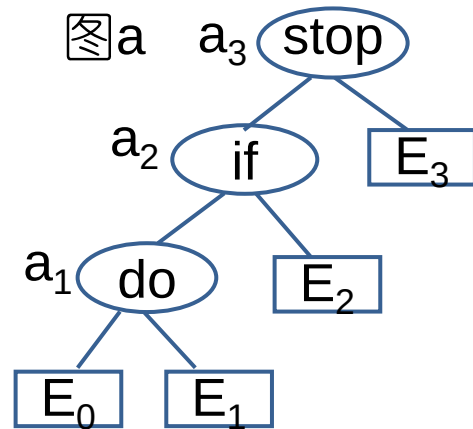
- 包含标识符集 $(a_1, a_2, a_3) = (\text{do}, \text{if}, \text{stop})$ 的二分检索树，有5种可能的结构，检索成本与树的结构相关。



检索成本也**与被检索的概率值**相关。

情景1: $P(i)=Q(i)=1/7$

情景2: $P(i)=(0.5, 0.1, 0.05), Q(i)=(0.15, 0.1, 0.05, 0.05)$



$$\text{cost}(a) = P(1)*3 + P(2)*2 + P(3)*1 + Q(0)*3 + Q(1)*3 + Q(2)*2 + Q(3)*1$$

$$\begin{aligned} \text{cost}(a-1) &= 3*1/7 + 2*1/7 + 1/7 + 3*1/7 + 3*1/7 + 2*1/7 + 1/7 \\ &= 15/7 \end{aligned}$$

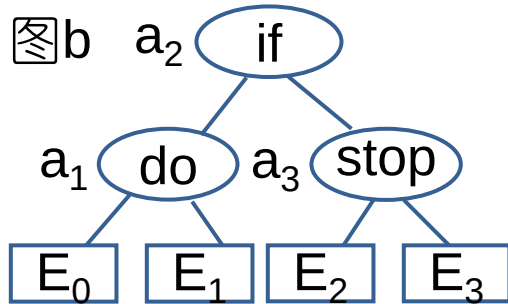
$$\begin{aligned} \text{cost}(a-2) &= 3*0.5 + 2*0.1 + 0.05 + 3*0.15 + 3*0.1 + 2*0.05 + 0.05 \\ &= 2.65 \end{aligned}$$



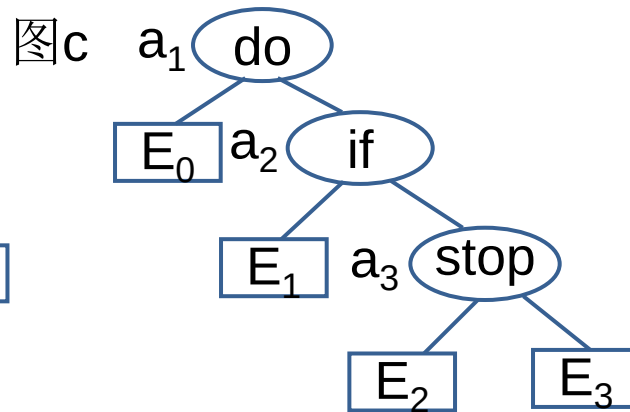
cost(a-1)=15/7
cost(a-2)=2.65

情景1: $P(i)=Q(i)=1/7$

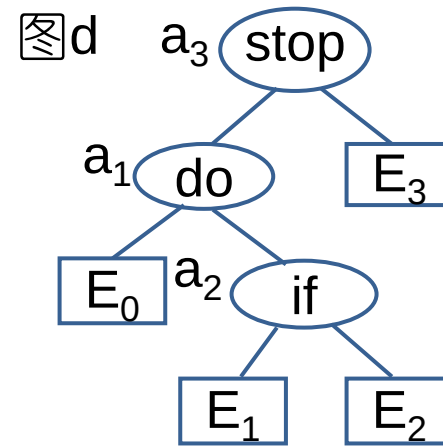
情景2: $P(i)=(0.5, 0.1, 0.05), Q(i)=(0.15, 0.1, 0.05, 0.05)$



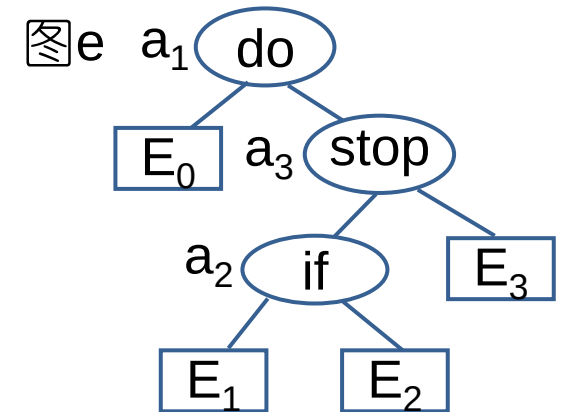
cost(b-1)=13/7
cost(b-2)=1.9



cost(c-1)=15/7
cost(c-2)=1.5



cost(d-1)=15/7
cost(d-2)=2.15



cost(e-1)=15/7
cost(e-2)=1.6

● 平均检索次数最小的树为最优二分检索树

● 比较五种树形结构

- 对于第1种情况, 内外结点概率相同, 图b所示的树最优。
- 对于第2种情况, 内外结点概率不同, 图c所示的树最优。

构造最优二分检索树问题描述

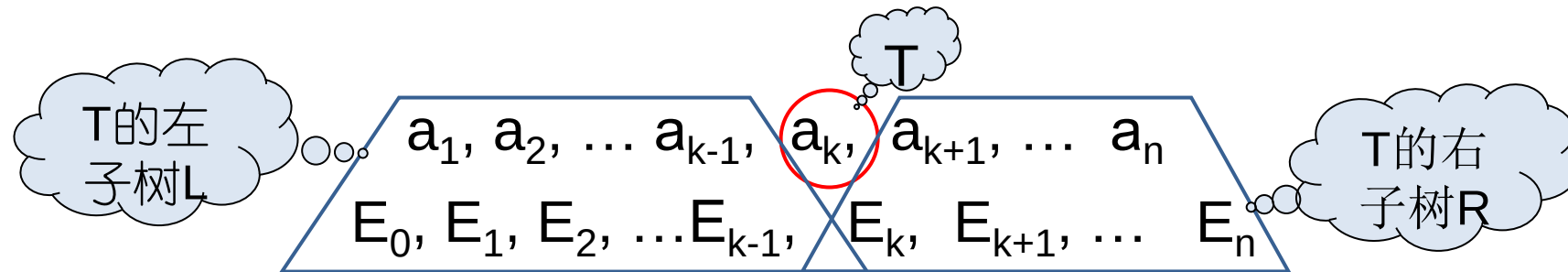
- 已知标识符集($a_1, a_2, \dots, a_n$), $a_1 < a_2 < \dots < a_n$; 已知检索概率 $P(i)$, $1 \leq i \leq n$, $P(i)$ 是对 $X = a_i$ 成功检索的概率, 还已知概率 $Q(i)$, $0 \leq i \leq n$, $Q(i)$ 是 $a_i < X < a_{i+1}$ 不成功检索的概率, 且 $\sum P(i) + \sum Q(i) = 1$.
- 构造预期检索成本最小的二分检索树, 即**最优二分检索树**。

二分检索树的预期成本为: $\sum_{1 \leq i \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$



问题分析及递推关系式

- 构造最优二分检索树，采用动态规划方法，把构造过程看成是一系列决策过程，即决策哪个结点为树根或子树的根。
- 对于 a_i , $1 \leq i \leq n$, 若 a_k 为 T 的根：



$$\text{左子树的成本: } \text{COST}(L) = \sum_{1 \leq i < k} P(i) * \text{level}(a_i) + \sum_{0 \leq i < k} Q(i) * (\text{level}(E_i) - 1)$$

$$\text{右子树的成本: } \text{COST}(R) = \sum_{k < i \leq n} P(i) * \text{level}(a_i) + \sum_{k \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$$

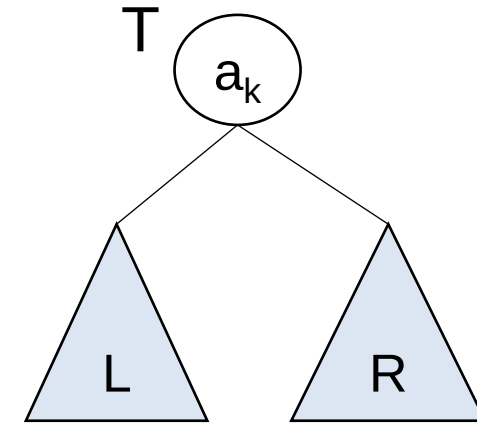
独立子树

整棵树T的成本:

$$\text{COST}(T) = \sum_{1 \leq i \leq n} P(i) * \text{level}(a_i) + \sum_{0 \leq i \leq n} Q(i) * (\text{level}(E_i) - 1)$$

$$= \boxed{\text{COST}(L)} + \boxed{\text{COST}(R)} + P(k)$$

$$W(0, k-1) = \boxed{+ \sum_{1 \leq i < k} P(i) + \sum_{0 \leq i < k} Q(i)} + \boxed{+ \sum_{k < i \leq n} P(i) + \sum_{k \leq i \leq n} Q(i)} = W(k, n)$$



令 $W(i, j) = Q(i) + \sum_{l=i+1}^j (Q(l) + P(l))$, 且有 $W(i, j) = W(i, j-1) + Q(j) + P(j)$

树T的预期成本可由左右子树的预期成本表达:

$$\text{COST}(T) = P(k) + \text{COST}(L) + \text{COST}(R) + W(0, k-1) + W(k, n)$$



动态规划求解步骤

- 证明满足最优性原理:
 - 若当前二分检索树T是最优的,则它的左右子树也一定是它们所对应的子问题的最优解(略)
- 设计递推式:
 - 问题最优解: 函数值表达最小期望成本
 - 问题边界和约束条件: 函数参数集表达标识符范围
 - 原问题和子问题关系: 最优性原理
- 自底向上方式实现程序
 - 求 $j-i=1$ 时的最优二分检索树
 - 求 $j-i=2$ 时的...
 - ...
 - 求 $j-i=n$ 时的最优二分检索树



递推关系式化简

- $C(i,j)$ 表示包含 $a_{i+1}, \dots, a_j$ 和 $E_i, \dots, E_j$ 的最优二分检索树的成本
- 根据最优性原理，如果 T 是最优的，那么 $COST(T)$ 必定是最小值， $COST(L)$ 和 $COST(R)$ 也必定是最小值。即 $COST(T)=C(0,n)$, $COST(L)=C(0,k-1)$, $COST(R)=C(k,n)$

- 树 T 的最小预期成本：每个根结点考察一遍

$$C(0,n)=\min_{1 \leq k \leq n} \{C(0, k-1)+C(k, n)+P(k)+W(0, k-1)+W(k, n)\}$$

- 将上式一般化，对于任何 $c(i,j)$ 有下式：

$$C(i,j)=\min_{i < k \leq j} \{C(i, k-1)+C(k, j)+P(k)+W(i, k-1)+W(k, j)\}$$

$$=\min_{i < k \leq j} \{C(i, k-1)+C(k, j)\}+W(i, j)$$

求解过程——实例

$$C(i,j)=\min\{C(i, k-1)+C(k, j)\}+W(i, j)$$

- 考虑初始时，内结点个数为0个， $j-i=0$: $C(i, i)=0$, $W(i, i)=Q(i)$, $0 \leq i \leq n$
- 基于公式计算 $j-i=1$ 的 $C(i, j)$, $j-i=2$ 的 $C(i, j)$, ..., $j-i=n$ 的 $C(i, j)$
- 记下每棵树 T_{ij} 的根 $R(i,j)$, $R(i,j)$ 是使 $C(i,j)$ 取最小值时的 k 值

例: $n=4, (a_1, a_2, a_3, a_4) = (\text{do}, \text{if}, \text{read}, \text{while})$, $P(1:4) = (3, 3, 1, 1)$; $Q(0:4) = (2, 3, 1, 1, 1)$

(1) $j-i=1$ 时, $0 \leq i < 4$

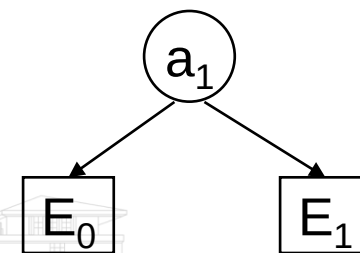
$$C(0,1) \quad W(0,1)=Q(0)+Q(1)+P(1)=2+3+3=8$$

$$C(0,1)=\min_{0 < k \leq 1} \{C(0,0)+C(k,1)\}+W(0,1)=0+8=8$$

$k=1$

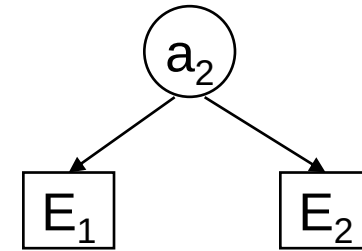
$$R(0,1)=k=1$$

$C(0,1)$

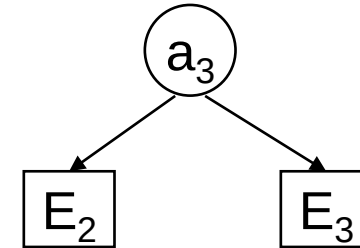


$$P(1:4)=(3,3,1,1); Q(0:4)=(2,3,1,1,1)$$

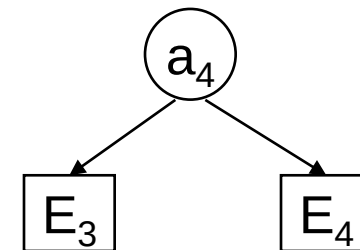
C(1,2) $W(1,2)=Q(1)+Q(2)+P(2)=3+1+3=7$
 $C(1,2)=\min_{1 \leq k \leq 2} \{C(1,1)+C(2,2)\}+W(1,2)=0+7=7$
 $R(1,2)=k=2$



C(2,3) $W(2,3)=3$
 $C(2,3)=3$
 $R(2,3)=k=3$



C(3,4) $W(3,4)=3$
 $C(3,4)=3$
 $R(3,4)=k=4$



$P(1:4)=(3,3,1,1); Q(0:4)=(2,3,1,1,1)$

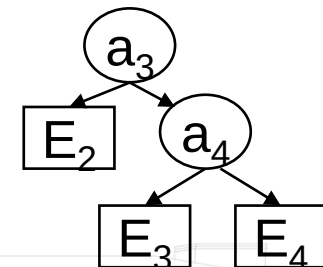
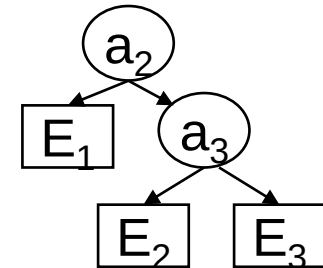
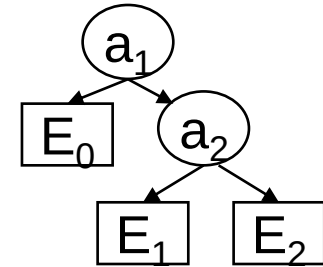
$W(0,1)=8$	$C(0,1)=8$	$R(0,1)=1$
$W(1,2)=7$	$C(1,2)=7$	$R(1,2)=2$
$W(2,3)=3$	$C(2,3)=3$	$R(2,3)=3$
$W(3,4)=3$	$C(3,4)=3$	$R(3,4)=4$

(2) $j-i=2$ 时, $0 \leq i < 3$

C(0,2) $W(0,2)=W(0,1)+Q(2)+P(2)=8+1+3=12$
 $C(0,2)=\min_{0 < k \leq 2} \{ \underbrace{C(0,0)+C(1,2)}_{k=1}, \underbrace{C(0,1)+C(2,2)}_{k=2} \} + W(0,2)$
 $=\min\{0+7, 8+0\} + 12 = 19$
 $R(0,2)=k=1$

C(1,3) $W(1,3)=W(1,2)+Q(3)+P(3)=7+1+1=9$
 $C(1,3)=\min_{1 < k \leq 3} \{ \underbrace{C(1,1)+C(2,3)}_{k=2}, \underbrace{C(1,2)+C(3,3)}_{k=3} \} + W(1,3)$
 $=\min\{0+3, 7+0\} + 9 = 12$
 $R(1,3)=k=2$

C(2,4) $W(2,4)=5 \quad C(2,4)=8 \quad R(2,4)=k=3$



$$P(1:4)=(3,3,1,1); Q(0:4)=(2,3,1,1,1)$$

(3) $j-i=3$ 时, $0 \leq i < 2$

$W(0,1)=8$	$C(0,1)=8$	$R(0,1)=1$
$W(1,2)=7$	$C(1,2)=7$	$R(1,2)=2$
$W(2,3)=3$	$C(2,3)=3$	$R(2,3)=3$
$W(3,4)=3$	$C(3,4)=3$	$R(3,4)=4$

$W(0,2)=12$	$C(0,2)=19$	$R(0,2)=1$
$W(1,3)=9$	$C(1,3)=12$	$R(1,3)=2$
$W(2,4)=5$	$C(2,4)=8$	$R(2,4)=3$

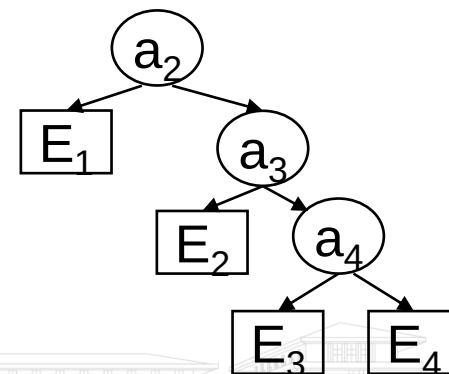
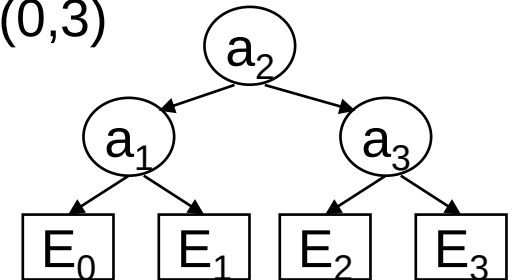
$C(0,3)$ $W(0,3)=W(0,2)+Q(3)+P(3)$
 $=12+1+1=14$

$$C(0,3)=\min_{0 \leq k \leq 3} \{ \underbrace{C(0,0)+C(1,3)}_{k=1}, \underbrace{C(0,1)+C(2,3)}_{k=2}, \underbrace{C(0,2)+C(3,3)}_{k=3} \} + W(0,3)$$

$$= \min\{0+12, 8+3, 19+0\} + 14 = 11+14=25$$

$R(0,3)=k=2$

$C(1,4)$ $W(1,4)=11$ $C(1,4)=19$ $R(1,4)=k=2$



$P(1:4)=(3,3,1,1); Q(0:4)=(2,3,1,1,1)$

$W(0,1)=8$	$C(0,1)=8$	$R(0,1)=1$
$W(1,2)=7$	$C(1,2)=7$	$R(1,2)=2$
$W(2,3)=3$	$C(2,3)=3$	$R(2,3)=3$
$W(3,4)=3$	$C(3,4)=3$	$R(3,4)=4$

$W(0,2)=12$	$C(0,2)=19$	$R(0,2)=1$
$W(1,3)=9$	$C(1,3)=12$	$R(1,3)=2$
$W(2,4)=5$	$C(2,4)=8$	$R(2,4)=3$

$W(0,3)=14$	$C(0,3)=25$	$R(0,3)=2$
$W(1,4)=11$	$C(1,4)=19$	$R(1,4)=2$

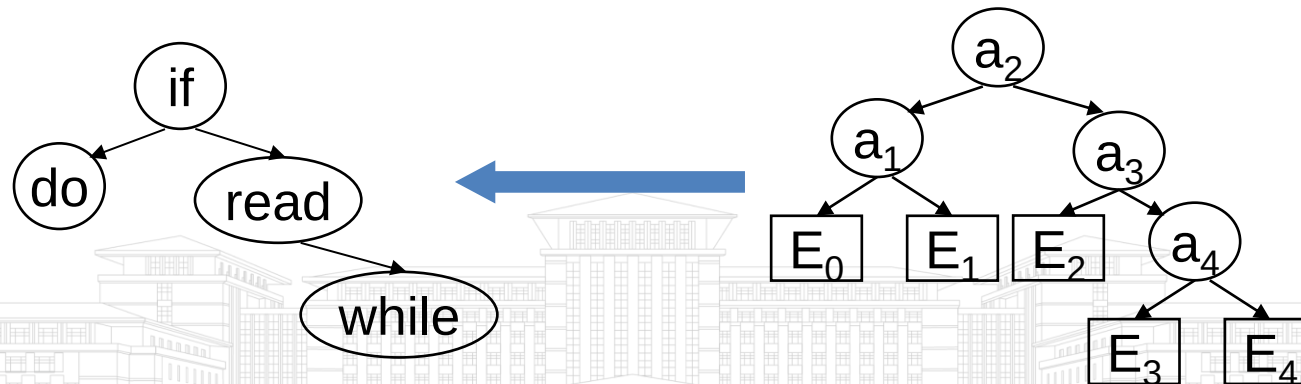
(4) $j-i=4$ 时, $0 \leq i < 1$

$$W(0,4)=W(0,3)+Q(4)+P(4)=16$$

$$C(0,4)=\min_{0 < k \leq 4} \{ \underbrace{C(0,0)}_{k=1} + \underbrace{C(1,4)}_{k=2}, \underbrace{C(0,1)}_{k=2} + \underbrace{C(2,4)}_{k=3}, \underbrace{C(0,2)}_{k=3} + \underbrace{C(3,4)}_{k=4}, \underbrace{C(0,3)}_{k=4} + \underbrace{C(4,4)}_{k=4} \} + W(0,4)$$

$$= \min\{0+19, 8+8, 19+3, 25+0\} + 16 = 16 + 16 = 32$$

$$R(0,4)=k=2$$



$j-i=0$

$W(0,0)=2$ $C(0,0)=0$ $R(0,0)=0$
 $W(1,1)=3$ $C(1,1)=0$ $R(1,1)=0$
 $W(2,2)=1$ $C(2,2)=0$ $R(2,2)=0$
 $W(3,3)=1$ $C(3,3)=0$ $R(3,3)=0$
 $W(4,4)=1$ $C(4,4)=0$ $R(4,4)=0$

$j-i=1$

$W(0,1)=8$ $C(0,1)=8$ $R(0,1)=1$
 $W(1,2)=7$ $C(1,2)=7$ $R(1,2)=2$
 $W(2,3)=3$ $C(2,3)=3$ $R(2,3)=3$
 $W(3,4)=3$ $C(3,4)=3$ $R(3,4)=4$

$j-i=2$

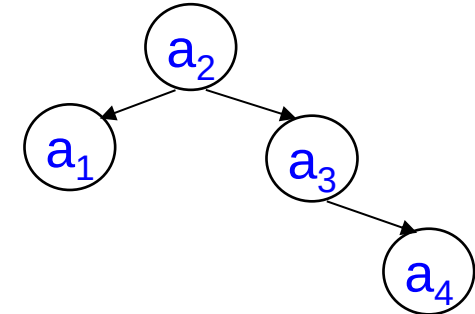
$W(0,2)=12$ $C(0,2)=19$ $R(0,2)=1$
 $W(1,3)=9$ $C(1,3)=12$ $R(1,3)=2$
 $W(2,4)=5$ $C(2,4)=8$ $R(2,4)=3$

$j-i=3$

$W(0,3)=14$ $C(0,3)=25$ $R(0,3)=2$
 $W(1,4)=11$ $C(1,4)=19$ $R(1,4)=2$

$j-i=4$

$W(0,4)=16$ $C(0,4)=32$ $R(0,4)=2$



$W(i,j)$	$j=0$	1	2	3	4
0	2	8	12	14	16
1		3	7	9	11
2			1	3	5
3				1	3
4					1

$C(i,j)$	$j=0$	1	2	3	4
0	0	8	12	25	32
1		0	7	9	19
2			0	3	5
3				0	3
4					0

$R(i,j)$	$j=0$	1	2	3	4
0	0	1	1	2	2
1		0	2	2	2
2			0	3	3
3				0	4
4					0

构造最优二分检索树算法

- 算法核心，三个矩阵——W, R, C:

初始值: $W(i,i) = Q(i)$; $W(i,i+1) = Q(i) + P(i+1) + Q(i+1)$;

$R(i,i) = 0$; $R(i,i+1) = i+1$;

$C(i,i) = 0$; $C(i,i+1) = Q(i) + P(i+1) + Q(i+1)$;



算法6.3 构造最优二分检索树算法

```
procedure OBST(P,Q,n)
real P(1:n), Q(0:n), C(0:n, 0:n), W(0:n, 0:n), integer R(0:n, 0:n),
for i←0 to n-1 do
  ( W(i,i), R(i,i), C(i,i)) ← (Q(i),0,0)
  (W(i,i+1), R(i,i+1) , C(i,i+1)) ←(Q(i)+Q(i+1)+P(i+1), i+1, Q(i)+Q(i+1)+P(i+1))
repeat
(W(n,n), R(n,n) , C(n,n) ) ←(Q(n),0,0)
for m← 2 to n do
  for i← 0 to n-m do
    j←i+m
    W(i,j) ← W(i,j-1)+P(j)+Q(j)
    k ←使{C(i,l-1)+C(l,j)}取最小值的l值
    C(i,j) ←C(i,k-1)+C(k,j)+W(i,j)
    R(i,j) ←k
  repeat
repeat
end OBST
```

优化方法: k 在 $[R(i,j-1), R(i+1,j)]$ 区间取值

时间复杂度分析

$$C(i,j)=\min_{i \leq k \leq j}\{C(i, k-1)+C(k, j)\}+W(i, j)$$

- $j-i=1$ 时, 计算了 n 个 $C(i,j)$
- $j-i=2$ 时, 计算了 $n-1$ 个 $C(i,j)$
- $j-i=m$ 时, 计算了 $n-m+1$ 个 $C(i,j)$
- 对于每个 $C(i,j)$, 要找出 $j-i$ 个量中的最小值, 因此, 对于 $j-i=m$ 的所有 $C(i,j)$, 总计算时间是 $O(m*(n-m+1))$
- 算法的总计算时间, 针对所有 $j-i=m (m=1,2,...,n)$ 求和:

$$\sum_{1 \leq m \leq n} (nm - m^2) = O(n^3)$$

D.E.Knuth的优化方法

- 求最优的k,可以把检索区间限制在 $R(i,j-1) \leq k \leq R(i+1,j)$ ，即

$$C(i,j) = \min_{R(i,j-1) \leq k \leq R(i+1,j)} \{C(i, k-1) + C(k, j)\} + W(i, j)$$

- 这种情况下，算法的计算时间为 $O(n^2)$ 。



6.7 矩阵链乘积问题

- 问题描述
- 问题分析
- 最优性原理证明
- 递推关系式设计
- 自底向上实现
- 算法描述
- 算法实例



问题描述及分析

- 矩阵链乘积问题

- 给定一个包含 n 个矩阵的序列，即矩阵链 $A_1, A_2, \dots, A_n$ ，计算它们的乘积 $A_1 A_2 \dots A_n$ ，使得计算代价最小，即所需的乘法运算次数最少。

- 乘法运算次数作为计算代价

$$T_{ij} = \sum_{k=1}^q A_{ik} B_{kj}$$

- 求矩阵 $A_{p \times q}$ 和 $B_{q \times r}$ 的乘积 $T_{p \times r}$
- 计算 $T_{p \times r}$ 中一个元素需要 q 次乘法； $p \times r$ 个元素，需乘法总次数为 $p \times q \times r$



- 矩阵运算顺序不同，代价不同
 - 例：矩阵链 $(A_1)_{p \times q}, (A_2)_{q \times r}, (A_3)_{r \times m}$
 - $(A_1 A_2) A_3$: $pqr + pr m$
 - $A_1 (A_2 A_3)$: $qrm + pqm$
 - 令 $p=10, q=100, r=5, m=50$ ，则乘法次数
 - $(A_1 A_2) A_3$: 7500
 - $A_1 (A_2 A_3)$: 75000
- 也就是，对矩阵链加括号的方式不同，运算代价不同

矩阵链乘积问题：寻找最优的计算次序（括号方案），使得总代价最小（乘法运算的次数最小）



动态规划法求解

- 矩阵链乘积
 - 是不是多阶段决策优化问题？
 - 是否满足最优性原理？
 - 设计递推关系式
 - 自底向上实现



多阶段决策特征及最优性原理证明

- 证明思想

- 要计算 $A_i A_{i+1} \dots A_j$ 的最小代价，每次要决策分割点的位置，一旦确定位置，原问题被分割成两个子问题。
- 假设最优解中最外层括号的分割点在 A_k 后，即 $(A_i \dots A_k) (A_{k+1} \dots A_j)$ ；最优解所给出的计算方案，对于子问题 $(A_i \dots A_k)$ 和 $(A_{k+1} \dots A_j)$ 也应该最优。



递推关系式设计

- 计算 $A_i A_{i+1} \dots A_j$ 的乘积, 其中 A_t 为 $p_{t-1} \times p_t$ 的矩阵, $t=i, \dots, j$
- 假设当前在 A_k 后分割, $i \leq k < j$, 原矩阵链被分割为两个矩阵链:

$$\begin{array}{c} A_i A_{i+1} \dots A_k \mid A_{k+1} \dots A_j \\ p_{i-1} \times p_k \mid p_k \times p_j \end{array} \quad \text{乘积矩阵的大小}$$

- 设 $m(i, j)$ 表示矩阵链 $A_i A_{i+1} \dots A_j$ 的最小代价, 则在 A_k 后分割的最小代价可表示为:

$$m(i, k) + m(k+1, j) + p_{i-1} p_k p_j$$

- 矩阵链 $A_i, \dots, A_j$ 的最小代价 $m(i, j)$ 是遍历每个分割点 $A_k (i \leq k < j)$ 后获得的最小值, 即

$$m(i, j) = \begin{cases} 0, & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1} p_k p_j\}, & i < j \end{cases}$$

自底向上实现

$$m(i, j) = \begin{cases} 0, & i = j \\ \min_{i \leq k < j} \{m(i, k) + m(k+1, j) + p_{i-1}p_kp_j\}, & i < j \end{cases}$$

- 根据公式求解 $A_1, \dots, A_n$ 的最小代价，即求解 $m(1, n)$
- 步骤如下(其中 $1 \leq i, j \leq n$):
 - 计算 $j-i=1$ 时的所有 $m(i, j)$ ，共计 $n-1$ 个
 - 计算 $j-i=2$ 时的所有 $m(i, j)$ ，共计 $n-2$ 个
 - ...
 - 计算 $j-i=n-1$ 时的所有 $m(i, j)$ ，共计1个，问题得解



算法实例

$n=6$

矩阵
规模

A_1	A_2	A_3	A_4	A_5	A_6
30×35	35×15	15×5	5×10	10×20	20×25

$m(i,j)$	$j=1$	2	3	4	5	6
$i=1$	0	15750	7875	9375	11875	15125
2		0	2625	4375	7125	10500
3			0	750	2500	5375
4				0	1000	3500
5					0	5000
6						0

$s(i,j)$	$j=1$	2	3	4	5	6
$i=1$	0	1	1	3	3	3
2		0	2	3	3	3
3			0	3	3	3
4				0	4	5
5					0	5

$m(1,2) > m(1,3) ?$

$$m(1,2) = A_1 \times A_2 = 30 \times 35 \times 15$$

$$m(1,3) = A_1 \times (A_2 \times A_3) = 35 \times 15 \times 5 + 30 \times 35 \times 5$$

$$(A_1 (A_2 A_3)) ((A_4 A_5) A_6)$$

算法6.4 矩阵链乘积算法

```
Procedure Matrix-Chain-Order(n)
//  $A_1$ 为 $p_0 \times p_1$ ,  $A_2$ 为 $p_1 \times p_2$ , ...  $A_n$ 为 $p_{n-1} \times p_n$ 
int m(1:n,1:n), s(1:n-1,1:n) //s记录分割点k
for i=1 to n-1 do //初始化
    (m(i,i),s(i,i))←(0,0), (m(i,i+1),s(i,i+1))←(pi-1pipi+1,i)
repeat
(m(n,n),s(n,n))←(0,0)
for w←2 to n-1 do //w=j-i表示问题规模, 即连乘的矩阵个数-1
    for i←1 to n-w do
        j← i+w
        k← 区间[i,j]中使{m(i,l)+m(l+1,j)+pi-1 pl pj}取最小值的l
        m(i,j) ← m(i,k)+m(k+1,j)+pi-1 pk pj
        s(i,j) ← k
    repeat
repeat
end Matrix-Chain-Order
```

6.8 小结

- 动态规划适用的问题特点
 - 多阶段决策优化问题
- 动态规划方法特点
 - 最优子结构性质
 - 重叠子问题性质
- 动态规划方法的时间复杂度受存储空间的影响
 - 备忘录：记录子问题最优解
 - 标记函数：记录对应最优解的决策序列



- 动态规划和分治法的区别
 - 分治法不具有重叠子问题性质
- 动态规划和贪心法的区别
 - 动态规划在做下一步决策时，依赖子问题的最优解
 - 贪心法在做下一步决策时，不考虑子问题结果，只顾眼前



●6.1 一般方法

- 掌握最优性原理、动态规划适用的问题特点、求解思想、动态规划的特点等基本知识。能掌握动态规划求解问题的一般过程。

●6.2 多段图

●6.3 货郎担问题

- 能掌握最优性原理证明和递推关系式的一般设计方法，动态规划算法的一般设计思想和时间复杂度分析方法。

●6.4 0/1背包问题

●6.5 可靠性设计

- 基于动态规划递推关系式演变算法，掌握序偶方法，深入理解存储空间对时间复杂度的影响。



- 6.6 最优二分检索树
- 6.7 矩阵链乘积问题

- 掌握复杂问题的递推关系式设计方法，掌握优化算法和降低复杂度的设计技巧
 - 0/1背包问题序偶对法：利用贪心法和估计值预判
 - 最优二分检索树：减少子问题范围
 - 矩阵链乘积问题：设计边界和标记

能够识别出适合动态规划的可计算性问题、独立设计算法和分析算法复杂度。





本章结束



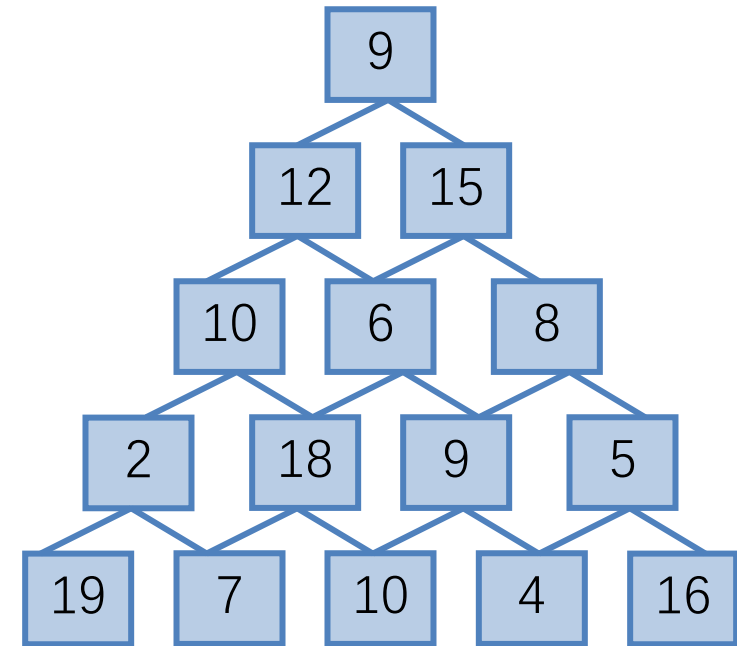
本章补充内容-1

- 动态规划vs. 贪心
- 动态规划vs. 分治



例-数塔问题

- 如图一个数塔，从顶部出发，在每个结点可向左走或向右走，一直走到底层。
- 求一条从顶到底的路径，且该路径上数值之和最大。



多阶段决策过程
求决策序列
找最优决策序列



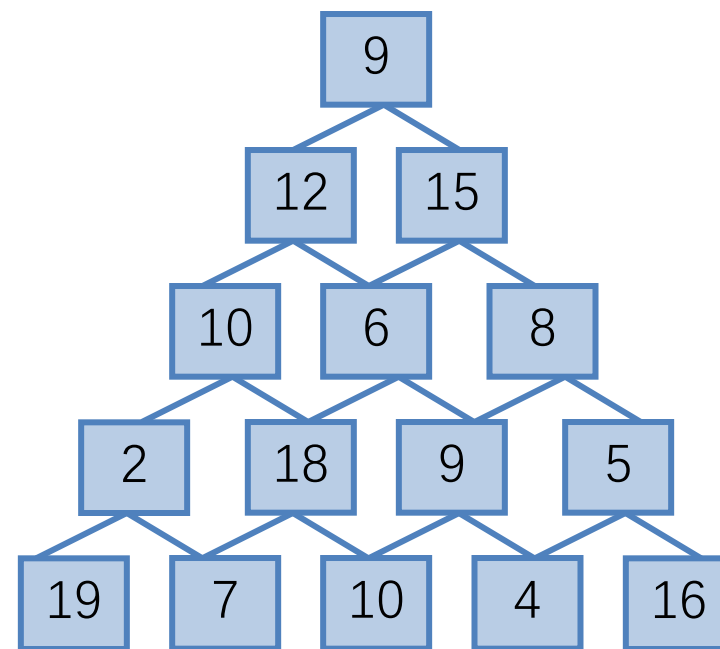
贪心方法的局部性

贪心方法：自上而下

$$9+15+8+9+10=51$$

贪心方法：自下而上

$$19+2+10+12+9=52$$

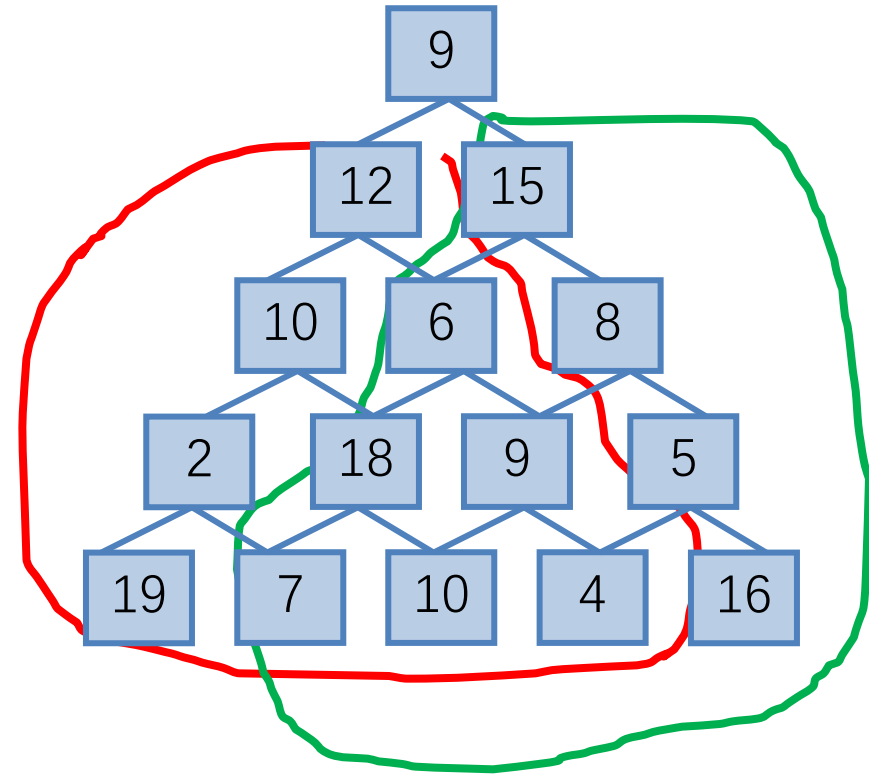


$$\text{最优解: } 9+12+10+18+10=59$$

贪心法做当前来说的最好决策，得到的不是全局最优解。

动态规划的全局性

- 要找到数塔最大和的前提是能看到数塔的**全貌**；
- 分别求两个**子数塔的最优解**，以得到全局最优解；
- 整个问题的最优解路径就是 $\max\{\text{根结点} + \text{子问题的最优解}\}$ 。



如何求子数塔的最优解？ 递归？ 分治法？

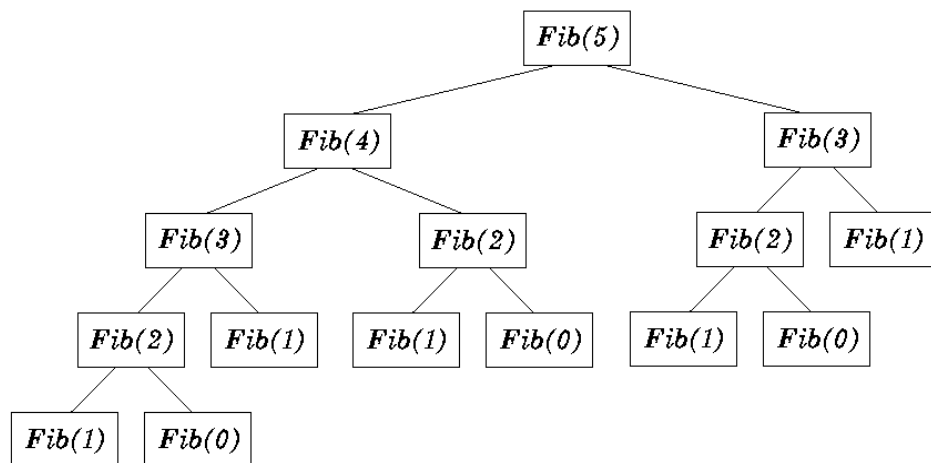
最优解： $9 + 12 + 10 + 18 + 10 = 59$

动态规划vs.分治法

- 相同之处：通过组合子问题的解，来求解原问题；
- 不同之处：
 - 分治法可将原问题划分为互不相交的子问题（即子问题不重叠），再将子问题的解组合，得到原问题的解；
 - 动态规划应用于子问题重叠的情况，不同的子问题具有公共的子子问题；
- 子问题重叠的处理
 - 若将分治法（或递归）应用于子问题重叠的情况，分治法会做许多不必要的工作，反复求解公共子子问题；
 - 动态规划法对每个子子问题，只求解一次，将其解保存起来，无需多次重复计算。

- 例如：递归求斐波那契数列，出现大量的子问题重复计算

$$f(i) = f(i - 1) + f(i - 2), i \geq 3$$



动态规划

```
void dp()
{
    f[1] = 1; f[2] = 1;
    for(int i = 3; i <= n ; i ++)
        f[i] = f[i-1] + f[i-2];
}
```

- 动态规划迭代求解，时间复杂度由指数阶变为线性阶

动态规划的求解特点

- 全局最优解。
- 高效处理：保持中间结果，避免重复计算。



本章补充内容-2

- 动态规划的其它例子
 - 任意两顶点间最短路径——Floyd算法
 - AOE网中求关键路径问题



任意两顶点间最短路径——Floyd算法

设邻接矩阵存储图，定义初始矩阵 $A^{(-1)}[i][j] = \text{Edge}[i][j]$;

1) 求 $A^{(0)}$, 即从 v_i 到 v_j 经由顶点可以是 $\{v_0\}$ 的最短路径长度;

2) 求 $A^{(1)}$, 即从 v_i 到 v_j 经由顶点可以是 $\{v_0, v_1\}$ 的最短路径长度;

...

n) 求 $A^{(n-1)}$, 即从 v_i 到 v_j 经由顶点可以是 $\{v_0, v_1, \dots, v_{n-1}\}$ 的最短路径长度;

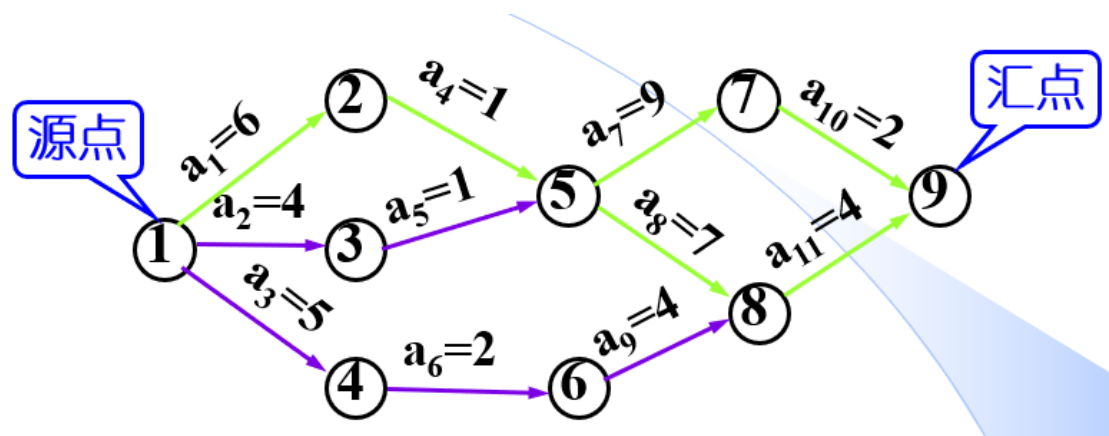
其中 $A^{(k)}[i][j] = \min \{ A^{(k-1)}[i][j],$

$$A^{(k-1)}[i][k] + A^{(k-1)}[k][j] \}, k = 0, 1, \dots, n-1$$



AOE网中求关键路径问题

有向无环的带权图中，用有向边表示一个工程中的各项活动，边上的权值表示活动的持续时间，用顶点表示事件，这样的有向图简称AOE (Activity On Edges) 网络。



- ◆ 完成整个工程至少需要多少时间？
- ◆ 在不影响整个工程进度的情况下，哪些活动可以适当延期？
- ◆ 为缩短完成工程所需的时间，应当加快哪些活动？

- 所有事件的最早发生时间(源点到各点的最长路径长度)

递推公式：// 按拓扑正序

$$ve(j) = \begin{cases} 0 & j=1 \\ \max\{ve(i) + \text{weight}(\langle i, j \rangle)\} & \langle v_i, v_j \rangle \in E(G), j=2, \dots, n \end{cases}$$

- 所有事件的最迟发生时间 (与各点到汇点的最长路径长度有关)

递推公式：//按拓扑逆序

$$vl(j) = \begin{cases} ve(n) & j=n \\ \min\{vl(k) - \text{weight}(\langle j, k \rangle)\} & \langle v_j, v_k \rangle \in E(G), j=n-1, \dots, 1 \end{cases}$$

- 根据ve, vl, 求活动的最早和最迟开始时间e和l
- 关键活动： $l(i) = e(i)$ 表示活动 a_i 是没有时间余量的关键活动。





本章补充内容结束

